

University of Potsdam

Institute of Computational Science

Bachelor of Science in Informatics/Computational Science

A Simulation Environment for Visualizing Foundational Artificial Intelligence Methods of Localization, Mapping, Planning and Control with Application in Robotic Systems.

Bachelor Thesis

of

Samuel Rodriguez

supervised by

Dr. Jana de Wiljes

2022

Contents

1 Zusammenfassung **2**

2 Introduction **2**

3 Theory & Implementation **3**

3.1 Main Structure 4

3.2 Localization 5

3.2.1 Basic Theory 5

3.2.2 Bayes Filter 6

3.2.3 Histogram Filters 7

3.2.4 Kalman Filter 1D 13

3.2.5 Kalman Filter 2D 15

3.2.6 Particle Filter 17

3.3 Motion Planning 20

3.3.1 A* 20

3.3.2 Dynamic Programming 23

3.3.3 Optimum Policy 25

3.4 Smoothing & Control 28

3.4.1 Path Smoothing 28

3.4.2 PID Controller 30

4 Discussion **35**

5 Conclusion **36**

6 Next Steps **36**

1 Zusammenfassung

Dieses Werk besteht aus der Implementierung und detaillierten Beschreibung einer Simulationsumgebung zur Visualisierung und dynamischen Interaktion mit grundlegenden Robotik-Algorithmen. Jeder Algorithmus wird durch eine eigene Schnittstelle abgedeckt, von denen es insgesamt 9 gibt. Diese Schnittstellen und die Methoden, die sie illustrieren, wurden in drei Hauptabschnitte unterteilt: 1. Lokalisierung: Schätzung der aktuellen Position des Roboters anhand von Sensoreingaben. 2. Bewegungsplanung: Suche nach dem besten Weg für ein gegebenes Ziel und Ausgangsposition unter Berücksichtigung der Kosten der verschiedenen Aktionen. 3. Glättung und Steuerung: Umwandlung einer diskreten Bahn in kontinuierliche Koordinaten und Senden von Befehlen an den Roboter, damit er der gewünschten Bahn folgt. In jedem Abschnitt wird eine allgemeine Einführung in die Prinzipien gegeben, die den Algorithmen der Gruppe gemeinsam sind, und für jede einzelne Schnittstelle wird eine Erläuterung der zugrunde liegenden Theorie gegeben, gefolgt von einer Beschreibung der Funktionsweise der Schnittstelle, die von Bildern begleitet wird. Der Fokus liegt auf der Veranschaulichung bestimmter Fälle, in denen die Simulationsumgebung im Gegensatz zu den bereits verfügbaren Alternativen besonders nützlich ist, und auf der Bereitstellung klarer, intuitiver Erklärungen zur Funktionsweise der jeweiligen Methoden. Abschließend wird eine Zusammenfassung der während des Entwicklungsprozesses aufgetretenen Herausforderungen gegeben, gefolgt von einem Abschnitt, in dem die Einschränkungen und möglichen nächsten Schritte erläutert werden.

2 Introduction

Thanks to the outstanding advances in Hardware and Software in the last decades, robotic systems are finally becoming more widespread after a long period of slow growth/stagnation. Moreover, further progress in Artificial Intelligence and Machine Learning have enabled robotic applications to expand from simple, repetitive or dangerous work to more sophisticated tasks involving complex, large scale human-robot collaboration and operation in increasingly unstructured environments that are inherently unpredictable. [1]. This trend is expected to accelerate over the next decades, making robotics one of the most important and rapidly growing fields both in research as in industrial and commercial applications.

Effective educational tools are fundamental to guarantee the proper training of the next generation of scientists and engineers that will work on this field. Nowadays, E-Learning has become a standard tool that students and professors use to complement more traditional teaching methods. However, there are currently multiple limitations for learners when it comes to acquiring basic knowledge in the field of robotics:

- **Lack of real-time interaction**. The traditional format of short videos, small projects and Q&A sections that most platforms offer is very static. There is plenty of evidence showing that learning by direct interaction greatly increases knowledge retention [2]. Furthermore, the inherent interactive nature of robotic systems can greatly benefit from a dynamic, interactive environment to more efficiently convey theoretical concepts and methods.
- **Lack of content covering the fundamentals in comparison to other fields**. There has been a huge surge in learning resources for AI and ML but there is still a big deficit in the availability of high-quality content

that covers the core principles in robotics. There exist already several platforms that offer robotic courses but they often take the theoretical underpinnings for granted (or make little emphasis in them) and go straight to the applications. The majority of theoretical information is included in papers or lecture series, which might be difficult to understand and absorb.

- **Lack of visualization tools**. Just as with interaction, visualization has shown to improve learning outcomes [3]. Most of the classes are delivered in a traditional manner presenting formulas and their derivation and asking students questions or posing coding challenges after each block. But even in cases when illustrations are presented, this are usually static images where the learner can't visualize the effects of changing parameters or modifying the environment.

As a contribution to help overcome the above mentioned limitations, this project is the implementation of a simulation environment for visualizing and interacting with foundational robotic algorithms in real time. The original motivation came from my personal experience while taking the course called "Artificial Intelligence for Robotics" offered by the online learning platform Udacity [1] and instructed by Sebastian Thrun. The course provides a great foundation for building such an environment since it explains both the theory and the implementations for most of the core algorithms used in robotic systems. However, the lack of visual and interactive components increase the effort necessary to develop better intuitions about the topics covered. Because of this I started developing my own tools to deepen my understanding on specific topics and eventually decided to extend the interface to cover all the topics taught in the course.

Regardless of the application domain, every robotic system needs to be able to sense its environment, make plans based on events, and use actuators to accurately perform actions based on the devised plan. Mobile robots also require the ability to create maps of its environment and locate themselves in it. Each of these abilities are covered by at least one interface in the simulation environment, and owing to the ubiquity of these abilities, this resource represents a valuable didactic tool for a wide range of domains.

3 Theory & Implementation

The sections described in this thesis are divided into three main groups according to the area that the included algorithms cover, namely ***Localization***, ***Planning*** and ***Smoothing & Control***. These sections follow the same order in which the topics are presented in the "AI for Robotics" course [2], this is also the order in which each interface was implemented. Before delving into each interface, the general structure of the simulation environment is described. Then each section is explained in detail by first providing a summary of the theoretical background of the topic and a description of the interface that illustrates the concept. The main focus will be given to both the basic principle of each algorithm and to the specific ways in which the interfaces provide new perspectives or facilitate the understanding of particular attributes of the respective concept being illustrated. Accordingly, the implementation details of the GUI will be for the most part ignored as they are specific to the library and language chosen and unlike

[1] https://www.udacity.com/course/artificial-intelligence-for-robotics–cs373
[2] https://www.udacity.com/course/artificial-intelligence-for-robotics–cs373

the core theoretical principles and visualization advantages do not transfer to other implementations. All the code is publicly available at [3]. The language used is Python 3.10+ and the main library used is PyGame [4].

3.1 Main Structure

The simulation environment is composed of a *main menu* (as can be seen in *Figure* 1) where the user can input his or her name and select a specific interface to open among a list of possible options. Once the interface has been selected, the user can click on *next* to go to a *secondary menu* (shown in *Figure* 2) which is specific for the interface selected and has five buttons with the following functions:

- ***About***: Show short explanation about how the algorithm works, how the environment is structured and how it can be used.
- ***Controls***: Show tabular description of the inputs that can be used to control the simulation and their effects.
- ***Settings***: Show list of parameters that can be adjusted before starting the simulation.
- ***Play***: Access the interface itself.
- ***Back***: Return to the main menu.

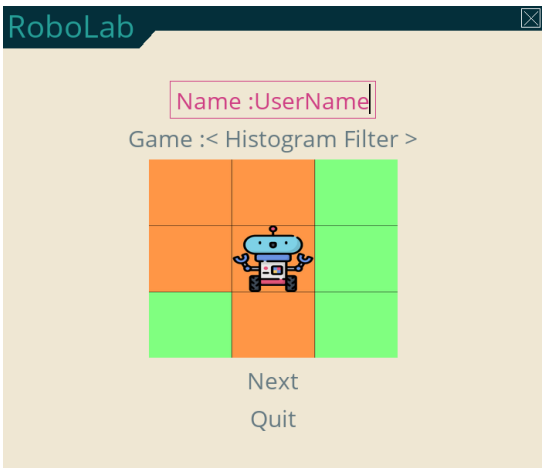


Figure 1: Main Menu.

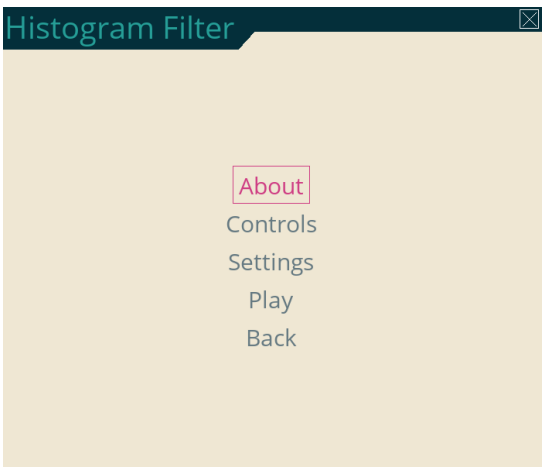


Figure 2: Secondary menu.

Since the contents of each of the sections contained in the *secondary menu* (i.e. *About*, *Controls*, *Settings* and *Play*) are different for every interface, figures and explanations for each will be presented in each case separately.

[3] https://github.com/Sam1320/robolab
[4] https://www.pygame.org/wiki/about

3.2 Localization

Probabilistic Robotics is a relatively recent approach to robotics which accounts for the uncertainty in the robot perception and actions. Instead of modelling states by using just single values, probabilistic methods represent information by using probability distributions over a space of possible hypotheses. By doing so, they can represent ambiguity and degree of belief in a mathematically sound way, enabling them to accommodate various sources of uncertainty. Moreover, by basing control decisions on probabilistic information, these algorithms perform nicely in the face of the various sources of uncertainty described above, leading to new solutions to hard robotics problems.[1] All of the algorithms presented in this section belong to the probabilistic framework. Another very important fact is that all of these localization methods **assume that a map of the environment is known**.

3.2.1 Basic Theory

This section provides a summary of the notation and basic concepts used to describe how the different localization algorithms work and their implementation. Mathematical derivations of the presented formulas and deeper explanations of probability theory are beyond the scope of this work, a good source that covers these topics in detail is the book *"Probabilisc Robotics"*[1] by Thrun et al.
To denote the probability that a random variable X takes the value x we write:

$$p(X = x) \tag{1}$$

The sum of probabilities for all possible outcomes is equal to one. This is denoted by:

$$\sum_x p(x) = 1 \quad \text{or} \quad \int_x p(x)dx = 1 \tag{2}$$

for the discrete and continuous case respectively. The joint distribution of random variables X and Y is written as:

$$p(x, y) = p(X = x \text{ and } Y = y) \tag{3}$$

If X and Y are independent from one another this joint distribution is given by:

$$p(x, y) = p(x)p(y) \tag{4}$$

If X and Y are dependent, then we are talking from conditional probability and it is denoted as:

$$p(x \mid y) = p(X = x \text{ under the condition that } Y = y) \tag{5}$$

As long as $p(y) \neq 0$ the conditional probability is given by:

$$p(x \mid y) = \frac{p(x, y)}{p(y)} \tag{6}$$

An interesting fact, which follows from the definition of conditional probability and the axioms of probability measures, is often referred to as theorem of total probability:

$$p(x) = \sum_y p(x \mid y)p(y) \quad \text{or} \quad p(x) = \int p(x \mid y)p(y)dy \tag{7}$$

Another useful implication of the definition is the *Bayes rule* linking the conditional probability $p(x \mid y)$ to $p(y \mid x)$:

$$p(x \mid y) = \frac{p(y \mid x)p(x)}{p(y)} \tag{8}$$

For our estimation problem we will model the environment using random variables. At each time-step t the environment is described by its state x_t which includes all the relevant variables that describe the current situation of the robot and its context. In a mobile system for example, the state x_t will contain the pose of the robot, which consists of its location and orientation, but it can also contain its velocity or the distance to certain features in the environment. The robot can either perform an *action* u or take a *measurement* z. The action u_t is the one performed between states x_{t-1} and x_t, while the measurement z_t perceives the state x_t.

We represent the probability of the environment currently being at state x_t as $bel(x_t)$ and denote this probability as the $belief$. This $belief$ depends only on the previous measurement and control [5] which can be written as:

$$bel(x_t) = p(x_t \mid z_t, u_t) \tag{9}$$

Since actions and measurements don't necessarily occur simultaneously It will prove useful to define the *intermediate belief* as the *belief* after an action has been taken and before incorporating the next measurement, this can be denoted as follows:

$$\overline{bel}(x_t) = p(x_t \mid z_{t-1}, u_t) \tag{10}$$

This also clearly depicts the way the robot interacts with its environment, namely through a constant interleaving of motion and perception.

3.2.2 Bayes Filter

Localization is the problem of estimating the coordinates and orientation of a robot from sensor data given a map of the environment. In the theory of stochastic processes, the filtering problem is a mathematical model for a number of state estimation problems in signal processing and related fields. The general idea is to establish a "best estimate" for the true value of some system from an incomplete, potentially noisy set of observations on that system.[4]. The algorithms described in this and the following sections belong to family of methods that try to solve the localization problem by using this filtering approach, they all calculate the belief updates from measurements and motion steps. The Bayes Filter is the most general one [1], and although there is no interface implemented for it, it makes sense to present it here since its basic principle also applies to the other methods and given its succinct, abstract nature it makes it easier to develop the intuition for how state estimation works.

[5] This is known as the *Markov Assumption.*

Algorithm 1: Bayes Filter

1 **function Bayes_filter** $(bel(x_{t-1}), u_t, z_t)$**:**
2 **forall** x_t **do**
3 $\quad \overline{bel}\,(x_t) = \int p\,(x_t \mid u_t, x_{t-1})\, bel\,(x_{t-1})\, dx_{t-1}$
4 $\quad bel\,(x_t) = \eta p\,(z_t \mid x_t)\, bel\,(x_t)$
5 **end**
6 **return** $bel(x_t)$

Figure 3: The algorithm represents a general template for recursive state estimation that is also used in the other filter methods. It calculates the new belief $bel(x_t)$ given the prior belief $bel(x_{t-1})$ together with the action u_t and measurement z_t between the two. To do so the belief update is split into a control update in line 3, changing the belief according to the action taken, and measurement update in line 4, adjusting the belief according to the measurement.

The filter works in two essential steps. First, it incorporates the control u_t into an intermediate belief $\overline{bel}(x_t)$ in line 3. This is obtained by integrating (summing) the product of the previous belief $bel(x_{t-1})$ and the transition probability $p(x_t \mid u_t, x_{t-1})$ over all states x_{t-1}. This step will be referred to as the control update or prediction.
The second step is called the measurement update. In line 4 the intermediate belief $\overline{bel}(x_t)$ is multiplied by the probability $p(z_t \mid x_t)$ of the measurement z_t being observed in state x_t . The resulting product needs to be normalized using a normalization constant η, giving us the final belief $bel(x_t)$. The algorithm needs to be initialized by an initial belief $bel(x_0)$. If one knows the value of x_0 with certainty, $bel(x_0)$ should be initialized with a point mass distribution that centers all probability mass on the correct value of x_0 , and assigns zero probability anywhere else. On the other hand, if one has no information about the initial value x_0 , $bel(x_0)$ may be initialized using a uniform distribution over the domain of x_0. This algorithm can only be implemented in the form stated here for very simple estimation problems. In particular, we either need to be able to carry out the integration in Line 3 and the multiplication in Line 4 in closed form, or we need to restrict ourselves to finite state spaces, so that the integral in Line 3 becomes a (finite) sum. [1]

3.2.3 Histogram Filters

The Histogram Filter is one of the simplest implementations of the Bayes Filter. It solves the problem of the infinite sum in Bayes Algorithm 3 by discretizing space into n intervals or bins, thus converting the motion update into a finite sum. For the two-dimensional case each bin represents an area and its assigned a probability corresponding to the $belief$ that the robot is located in that area. In accordance to Equation 2 the sum of the values of all the bins equals one. Intuitively this means that it is certain that the robot is located somewhere in the world.

The motion update in line 3 assigns an *intermediate probability* [6] $\overline{p}_{k,t}$ to each region x_k based on the sum of the transition probabilities from each region x_i to region x_k weighted by the probability that the robot was located in region x_i in the previous time-step t_{i-1}. For example, if we divide space in 3 bins denoted as regions x_1, x_2 and x_3, then the probability that the robot is located in region x_2 at time t after motion u_t is given by the probability that motion u_t causes the robot to go from x_1 to x_2 times the probability that the robot was in x_1 at time $t-1$, plus

[6] Analog to *intermediate belief*, *intermediate probability* is the one after motion and before the next measurement.

Algorithm 2: Histogram Filter

1 **function Histogram_filter** $(\{p_{k,x_{t-1}}\}), u_t, z_t)$**:**
2 **forall** k **do**
3 $\quad \overline{p}_{k,t} = \sum_i p\left(x_k \mid u_t, x_i\right) p_{i,t-1}$
4 $\quad p_{k,t} = \eta p\left(z_t \mid x_k\right) \overline{p}_{k,t}$
5 **end**
6 **return** $\{p_{k,t}\}$

Figure 4: This algorithm implements an iteration of the Histogram Filter. The belief has been discretized into a finite set of probabilities $\{p_{k,t-1}\}$ for regions $\{x_k\}$. In the algorithm we apply the standard Bayes Filter steps for a non-continuous state space to this set and arrive at the new set of probabilities $\{p_{k,t}\}$, using the control u_t and measurement z_t [5].

the probability that motion u_t causes the robot to go from x_3 to x_2 times the probability that the robot was in x_3 at time-step $t-1$. In reality most of the transition probabilities $p(x_k \mid u_t, x_i)$ are going to be zero, since the robot can only move a certain distance d away from its previous position, meaning that all transitions from regions further away than d from the robot will be zero.

The measurement update in line 4 integrates the measurement z_t by taking the *intermediate probability* $\overline{p}_{k,t}$ (after motion) and multiplying it by the probability of observing z_t given that the robot is in region x_k and then normalizing it by η to make sure that the probabilities add to one. In our previous example of the world divided in three regions x_1, x_2 and x_3, lets say that the regions have colors *red, green* and *red* respectively, and the color of the region is the only thing the robot can sense when it makes a measurement. Furthermore lets say that the sensor is not perfect and we know the probabilities it has of sensing *red* when the region is actually *green* or sensing *green* when the region is *red* and so on. Then if the robot senses *red*, the probability of it being in region x_2 is going to be given by the probability that it is in region x_2 after motion u_t multiplied by the probability that it senses *red* given that the region is *green*. In the general case, the perceived features can be anything depending on the sensors the robot has, from distances given by ultrasonic [6] or lidar [7] sensors to images given by cameras [8] or temperature measured by thermometers [9].

Consider now a learner getting familiar for the first time with the theory of Histogram Filters and then trying to understand its behaviour at runtime by implementing 4 in some programming language and printing out the updated probabilities to the console. One of the main difficulties comes from the fact that the probabilities for each region become smaller as the number of regions increases (either due to a bigger map or increased resolution in the current map). So although for our simple example the probabilities might look something like this: $[0.1, 0.8, 0.1]$, for bigger number of regions, parsing the probabilities becomes a hard task very quickly. It has been shown that colors greatly and positively impact retention when learning and creates less cognitive load when parsing information [10]. Considering this, one promising improvement is to map the probabilities from numeric values to a spectrum of colors going from dark blue representing low probabilities to dark red representing high probabilities and then using a graphical user interface to offer a colored grid view depicting the world and the current probabilities as we can see in Figure 5.

In this interface the user has the possibility to perform motion updates in directions *up, down, left* and *right*,

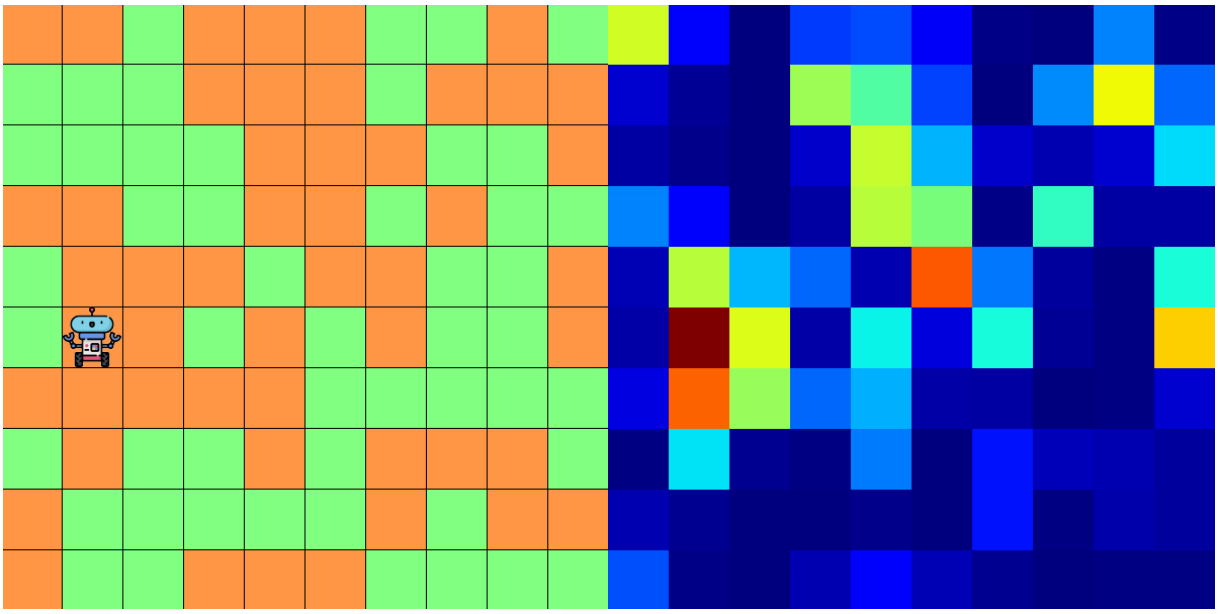


Figure 5: Histogram Filter simulation of a 2D grid-map where each cell can be either green or orange. On the left side there is a view of the world and the actual location of the robot. On the right side the 2D histogram represents the probabilities (*i.e. belief)* that the robot is located at each cell, where dark blue is the lowest and dark red the highest probability value respectively. One can notice how the grid cell with the highest probability corresponds to the location of the robot and that the lowest probabilities (dark blue) are distributed among the green cells, this is because the last measurement of the robot was *red*.

and also perform measurement updates by clicking the S key ("S" stands for *sense*) as depicted in Figure 6. The world is cyclical, meaning that if the robot is on the rightmost column of the grid and moves one cell to the right it is going to land on the leftmost column. The same principle applies for the upper- and lowermost rows and moving up or down. Just as with our simplified 3 bins example, the robot can only sense the color of the current cell it is in. It is worth noticing the possibility to give motion and sensing commands separately, since it is generally not possible (or not easy) to do so in the available visualization or simulation environments like Rviz [7] or Gazebo [8]. This is because they are meant for state estimation itself and not to visualize the interplay and consequences performing motion and sensing commands in arbitrary order. This environments allow to simulate or visualize inputs from actual robots. Normally in this setting the robot is constantly receiving information from its sensors and executing motion commands which are given either by the user or by its planning module after a goal has been set by the user.

To illustrate the advantage of being able to separately perform motion and sensing commands lets look at two specific examples, one where the robot only senses and one where it only moves.
Before describing the "only motion" case, it is useful to understand what is to be expected after motion commands. The motion update results in a shift of the probabilities in the direction of motion. If noise in the actuators exists, for example if the voltage sent to the wheel motors doesn't make them spin the exact desired amount, then this

[7] http://wiki.ros.org/rviz
[8] https://gazebosim.org/home

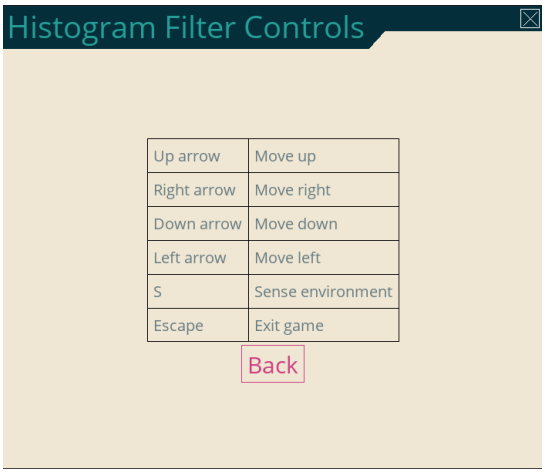


Figure 6: Histogram Filter Controls.

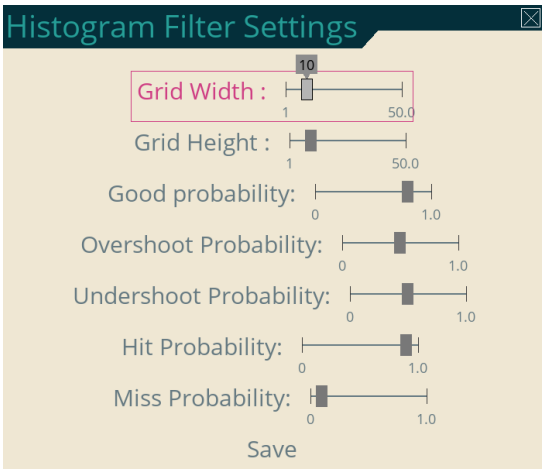


Figure 7: Histogram Filter Settings.

update is also going to *increase* the uncertainty of the robot's location. Conversely, when there is no noise during motion, then the uncertainty stays the same. For example, in our 3 bins example, suppose we are *80%* certain that the robot is located in region x_1 and *20%* certain that it is in x_2 (which means $p_1 = 0.8$, $p_2 = 0.2$ and $p_3 = 0$). Furthermore lets say that there is **no noise** in the motion of the robot, i.e. it executes the command with a success probability of *100%*. If we give it the motion command to go *Right* one cell, our probabilities will then simply *shift* one region to the right and our uncertainty will stay the same. So we would be left with the new probabilities $p_1 = 0$, $p_2 = 0.8$ and $p_3 = 0.2$. But in the case that there **is noise**, so that the robot only succeeds with *80%* probability and *20%* of the time it fails to move and stays in place. If our initial probabilities are lets say $p_1 = 1$, $p_2 = 0$ and $p_2 = 0$, meaning that in this case we are *certain* that the robot is in region x_1, and we give the same command as before to go *Right*, then the new probabilities will be $p_1 = 0.2$, $p_2 = 0.8$, and $p_3 = 0$. So our probability *shifted* to the right and we are *not certain* anymore about the location of the robot because of the noise introduced by the motion taken. In reality every robot has some degree of noise in its actuators, which means that every time motion is executed the uncertainty about its current state is going to *increase*. This is clearly depicted in the transition from the state shown in Figure 8 to the state shown in Figure 9. In Figure 8 we have the case where the robot, after exploring the environment (i.e. moving and sensing), is fairly certain of its location. Then it executes several *left* motion commands with no measurement, resulting in the new location and $belief$ shown in Figure 9, where the original high probability in cells (4, 4) and (4, 6) [9] has been spread out along the respective rows 4 and 6. In other words, before moving the robot "believed" with high probability to be in cell (4, 4) or (4, 6) and then after moving left for a while without sensing its environment, the robot now "believes" that it is located *somewhere* along the rows where cell (4, 4) and (4, 6) where located. If the robot had moved *down* instead, then we would see a red vertical stripe in the 4th column on Figure 9.

In contrast to (noisy) motion making the robot more *uncertain* of its location, sensing its environment makes the robot more *certain* of where it is. This is why measurement updates normally come immediately after motion

[9] There is also high probability of the robot being located in cell (3, 4), (4,7) and others, but for simplicity only two cells are considered.

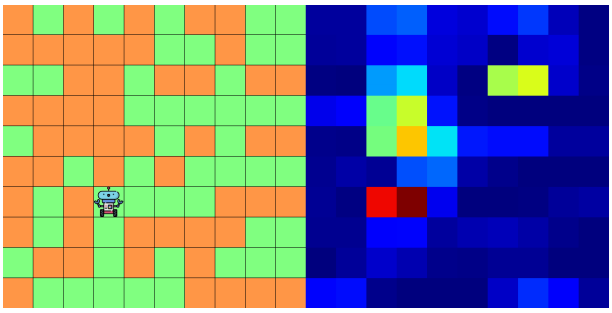


Figure 8: Histogram Filter interface with one measurement update after multiple previous motions and measurement actions. This actually reflects the typical case when navigating an environment.

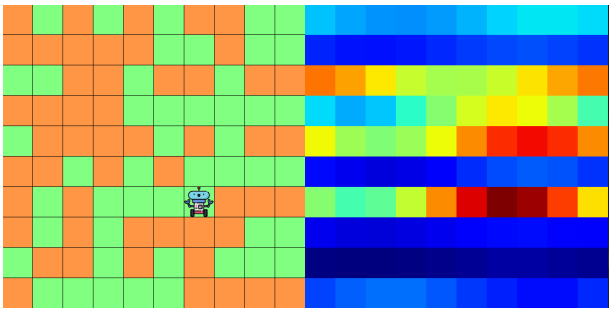


Figure 9: Histogram Filter interface with multiple motion updates not coupled with measurement updates. The probabilities are spread across the direction of movement due to the noise introduced during motion.

updates. However, only sensing the environment is not enough for localization. If the environment has very similar or identical features located at different locations, then measuring such a feature will only be enough to discard locations where that feature is not included, but it will not help resolve the question "on which location, from the ones that have that feature, is the robot?". In order to solve this ambiguity, it is necessary to move around and keep sensing in order to find more features and thus collect more information about the actual current location. As usual, lets illustrate this with our 3-regions-example. Suppose that our sensor and actuators have no noise and the colors of the x_1, x_2 and x_3 regions are *red*, *green* and *red* respectively. Moreover, there is no information about the current location, so that $p_1 = p_2 = p_3 = 0.\overline{33}$. If the robot senses *red* then it is going to discard the possibility of being in the *green* region x_2, but the probability will still be evenly split between regions x_1 and x_3. So the updated $belief$ is now $p_1 = 0.5, p_2 = 0$ and $p_3 = 0.5$. In order to resolve this ambiguity the robot moves *right* 2 times, updating its $belief$ to $p_1 = 0, p_2 = 0.5$ and $p_3 = 0.5$ (lets remember the world is cyclical). Then if it senses, the color measured is going to be again *red*, but this time the updated probabilities are $p_1 = 0, p_2 = 0$ and $p_3 = 1$. So after performing 2 motion steps and subsequently one measurement there is no more ambiguity in the robot's location estimate. Generally speaking, the more distinctive the measured feature is, the more information it provides about the location of the robot. This is why it is difficult to perform localization in environments which are monotonous, very symmetric, or texture-less [11]. This problem is depicted in Figures 10 and 11, where the robot first moves without sensing and hence maintains its absolute uncertainty about its location, as the uniform dark blue on the right hand side of Figure 10 shows. Then it moves one cell to the left and subsequently performs a measurement update resulting in the state shown in Figure 11. As can be seen, the red pattern in the histogram perfectly matches the locations of the red cells on the grid, denoting that the robot "knows" it is on a red region but doesn't know which. Note that no matter how many measurements are performed, motion is first necessary to be able to then gain more information about the location of the robot with subsequent measurements.

The main purpose of the probabilistic robotics framework is to be able to handle *uncertainty*, the main source of which is the *noise* in its sensors and actuators. If all input and outputs where noiseless, i.e. perfect, then the robot wouldn't need to deal with probability distributions but with scalar values which would represent the ground truth.

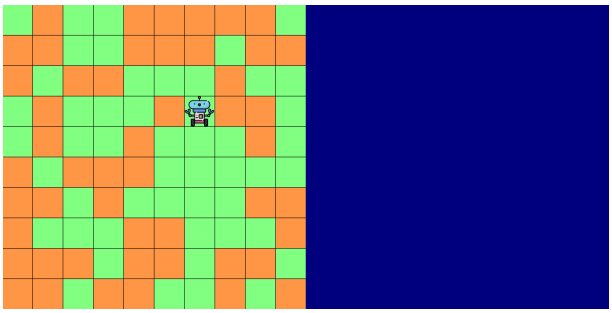


Figure 10: Histogram Filter interface where only motion actions have been taken since initializing the environment. The uniform blue on the right hand side denotes the fact that in order to gain information about the environment measurement updates are necessary.

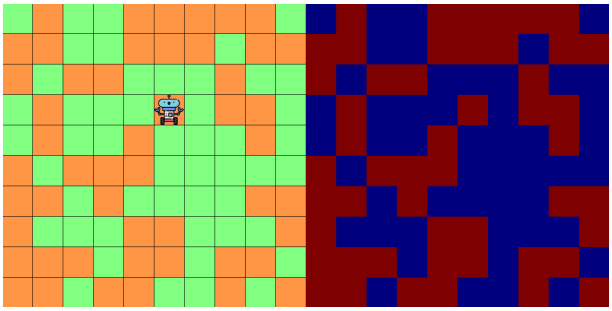


Figure 11: Histogram Filter interface where only measurement actions have been taken after several motion commands. The uniform red pattern on the right side matching the red cells pattern on the left shows the ambiguity that comes from similar (in this case identical) features being sensed.

Evidently this is an unrealistic assumption, hence there is the need to model the noise in each device. In real robots the noise can be estimated by testing the sensors and actuators and comparing the actual values with the measured ones and the actual displacement with the given motion command [10]. The advantage of dealing only with simulated robots is that one can set the noise values arbitrarily and observe the behaviour of the system. For that purpose, the settings section of the Histogram Filter interface allows to set both the motion and the sensor noise as shown in Figure 7. The *Good Probability* represents the probability that the motion taken corresponds with the motion command given, e.g. if we tell the robot to move *one cell to the right* then it moves exactly that amount in that direction. With probability of $(1 - \textit{Good Probability})$ the motion taken won't correspond with the command given, in this case there are two possibilities: Either the robot moves *one extra cell* with probability *Overshoot Probability* or the robot *does not move* with probability *Undershoot Probability*. It is worth noticing that $\textit{Undershoot Probability} = (1 - \textit{Overshoot Probability})$ and that $\textit{Good Probability} + (\textit{Overshoot Probability} + \textit{Undershoot Probability}) = 1$. This means that, on the settings 7 one first has to set the *Good probability* and then the remaining $(1 - \textit{Good Probability})$ is split between the *Overshoot* and *Undershoot Probabilities*, where moving one slider automatically adjusts the other one to make the probabilities add up to 1. For example, if we set *Good Probability* to 0.9 and *Overshoot Probability* to 0.8 then *Undershoot Probability* will be automatically set to 0.2. This means that *90%* of the time the robot will move correctly, and of that remaining *10%*, *8%* of the time the robot will overshoot and *2%* it will not move.

This same logic applies to the sensor which measures the color of the current cell. The *Hit* and *Miss Probability* sliders shown in Figure 7 represent the probabilities of making a good or a false measurement, in this case it means sensing the right color. So for $\textit{Hit Probability} = 0.9$ and $\textit{Miss Probability} = 0.1$ the robot will sense correctly *90%* of the time (i.e. it will sense *red* if the color of the current cell is *red*) while *10%* of the time it will sense the other color. All of this parameters change the behaviour of the simulation, in particular, the more noise there is in the system the longer it will take the robot to have a good estimate of its actual location.

A lot of the principles described and illustrated in this section also apply to the other filtering algorithms. For

[10] This applies for example when trying to estimate the noise in the motors that rotate the wheels in a mobile robot

this reason, particular emphasis was given into providing multiple detailed examples to help understand the core dynamics intuitively. On the next sections, it is assumed that the reader already understands this basic principles, specially the fact that *motion increases* and *sensing decreases uncertainty*, and the implications thereof. So the focus hereafter will be set into showing the differences of the respective methods to the one presented here.

3.2.4 Kalman Filter 1D

One of the biggest disadvantages of Histogram filters is that their memory requirement scales *exponentially* in the number of dimensions of the state space. Therefore it is very unpractical to use this approach when dealing with big maps or when high resolution in state estimation is required. The other limitation of Histogram Filters is that they work on discrete space, which doesn't correspond to the continuous space on which physical robots operate. This problem is not present on Kalman Filters which instead of discretizing space and representing the *belief* by assigning a probability to each region, they represent the current location estimate with a normal distribution, i.e. one *mean* value for each dimension and an n by n covariance matrix to represent the uncertainty, where n represents the number of dimensions. Since the biggest data structure needed to be kept track of is the covariance matrix (as opposed to a big grid of all the regions), the memory requirement scales *quadratically* in the number of dimensions of the state space, making it a very efficient method for state estimation.

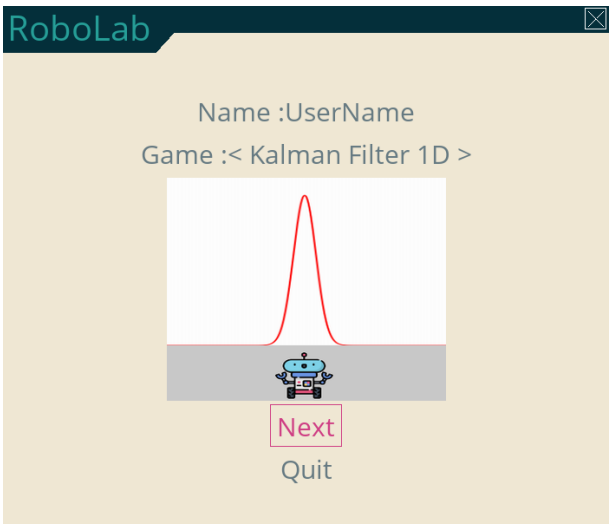


Figure 12: Kalman Filter 1D interface main menu.

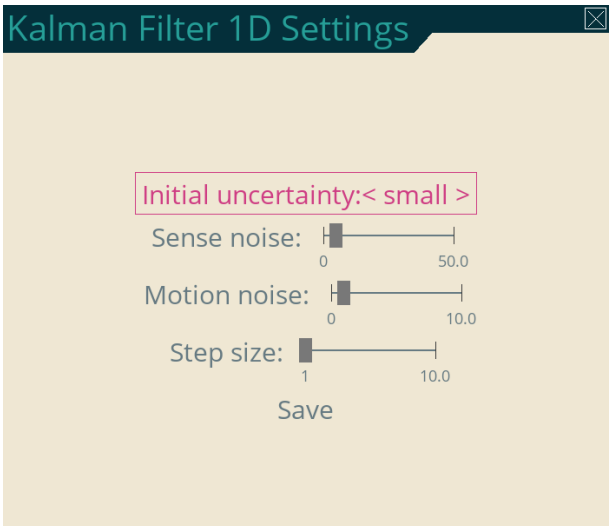


Figure 13: Kalman Filter 1D interface settings.

Normally, Kalman Filters are introduced with formulas and concepts that cover the general multidimensional case. However it is easier to first understand the principles behind it considering only one dimension and then see how it can be adapted to generalize for any number of dimensions. In the one dimensional case, only 2 variables are necessary to represent the belief function, this are the mean μ and the variance σ of the 1 dimensional normal distribution. Just as with the Bayes and Histogram filters, the motion update will involve performing a summation and the measurement update will be calculated by a product. For example, lets imagine a robot that can only move

along a track that fixes its movement to one dimension only and all it can sense is its distance to one of the ends of the track. Suppose that the length of the track is N meters and the state of the robot is represented by its position $0 \leq x \leq N$ along the track, so $x = 0$ means that the robot is at the beginning and $x = N$ means that the robot is at the end of the track. Initially the robot has no information of its location, thus the initial estimate of the position μ might be given the some random value $0 \leq x \leq N$ with a high standard deviation $\sigma^2 = N$. Visually this will draw a very flat normal distribution centered at x which represents the robot's prior *belief*. The measurement update, which is analog to line 4 in the Bayes Filter 3, consists of multiplying two Gaussians, one representing the prior belief and one representing the measurement. This measurement Gaussian is denoted by a mean ν, which is the value returned by the distance sensor, and a variance of r which corresponds to the noise of the sensor. When the robot senses, it will update its parameters to new μ' and $\sigma^{2'}$ values as shown in equations 11 and 12.

$$\mu' = \frac{r^2\mu + \sigma^2\nu}{r^2 + \sigma^2} \tag{11}$$

$$\sigma^{2'} = \frac{1}{\frac{1}{r^2} + \frac{1}{\sigma^2}} \tag{12}$$

There are two main properties to notice in this equations. One is the fact that the new mean μ' is basically a weighted sum of the prior *belief* μ and the measurement mean ν, the crucial thing to notice is that the weight for each mean is the standard deviation *of the other Gaussian*. This means that the mean of the Gaussian with the lowest variance is going to get the bigger weight. As a result, the new mean is going to be closer to the more "certain" Gaussian. In other words, if the uncertainty in the position of the robot is lower than the noise in the sensor, the new mean after the measurement update is going to be closer to the prior *belief* than to the value returned by the sensor. Conversely if the robot is unsure of its current location and has a very good sensor with low noise, the new *belief* is going to be closer to the sensor value. The other property to notice is that the new variance $\sigma^{2'}$ is going to be strictly smaller than both σ^2 and r^2 [11]. This might seem surprising because it means that if we multiply one peaky Gaussian with a very flat Gaussian, the product is going to be an *even peakier* Gaussian. Intuitively this is the case because we are gaining information, the combination of the two Gaussian has more information content as either Gaussian in isolation [1]. This has a very important implication: no matter how noisy the sensors we use, the information provided by them can always be useful. This fact is the base principle behind sensor fusion, a method for combining information from multiple different sensors [12].

The motion update, analog to line 4 of the Bayes filter 3, involves simply shifting the mean in the direction of motion and adding the two variances as can be shown in equations 13 and 14.This reflects the fact that, as observed in the Histogram Filter, the motion update *shifts* the *belief* and *increases* the *uncertainty*.

$$\mu' = \mu + \nu \tag{13}$$

[11] This is true because $\sigma^{2'} = \frac{1}{\frac{1}{r^2} + \frac{1}{\sigma^2}} < \frac{1}{\frac{1}{r^2}} = r^2$ and analog for σ^2

$$\sigma^{2'} = \sigma^2 + r^2 \tag{14}$$

Figures 14, 15 and 16 show three screenshots of the same run of the Kalman Filter 1D interface which illustrates a scenario similar to the one in the example presented above of the robot moving along a track and measuring its distance to the end of the track. In Figure 14 we can see the initial condition before any action has been taken and thus the uncertainty is very high as depicted by the wide Gaussian. Then the robot senses and updates its *belief* which results in a peaky Gaussian representing lower uncertainty as before the measurement as can be seen in figure 15. After this, the robot moves to the right, thereby increasing its uncertainty and shifting the *belief* as shown in Figure 16.

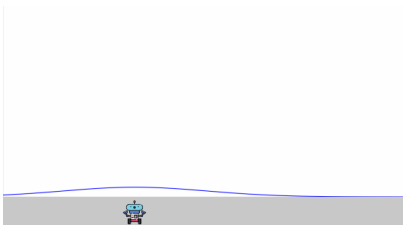


Figure 14: Kalman Filter 1D interface where the robot has not perform any motion or measurement yet. The very wide gaussian represents the high uncertainty at initialization.

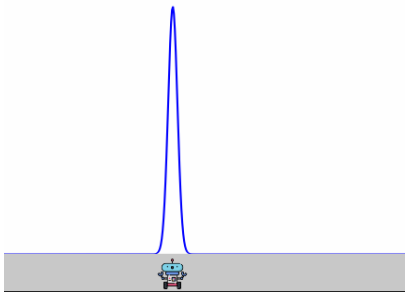


Figure 15: Kalman Filter 1D interface where the robot has just performed a measurement update. The very narrow gaussian represents the increased certainty due to integrating sensor information.

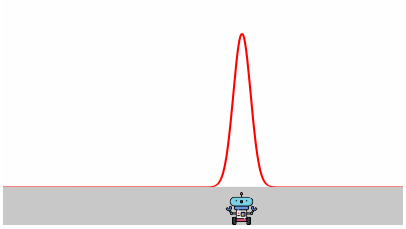


Figure 16: Kalman Filter 1D interface where the robot has performed motion after sensing. The red color denotes the fact that the certainty has decreased due to the noise introduced by the robot's motion.

3.2.5 Kalman Filter 2D

The generalization from one to n dimensions requires the introduction of formulas obtained from executing the integrations and products of the Bayes Filter using multivariate Gaussian distributions. However, the underlying principles stay the same. The main difference for n dimensions, is that the *belief* function is now going to be given by a Gaussian consisting of a vector $\mu = \begin{pmatrix} \mu_0 & \cdots & \mu_n \end{pmatrix}$ representing the mean of the state estimate, and a $n \times n$ covariance matrix Σ representing the uncertainty or noise in the system:

$$bel(x_0) = \mathcal{N}(\mu_0, \Sigma_0)(x_0). \tag{15}$$

In Kalman filters the transition probability $p(x_t|u_t, x_{t-1})$ must be a linear function and it is described by:

$$p(x_t|u_t, x_{t-1}) = \mathcal{N}(A_t x_{t-1} + B_t u_t, R_t)(x_t) \tag{16}$$

where A_t and B_t are matrices representing the state transition and control-input models respectively and R_t is the covariance matrix which describes the control's noise. The measurement probability must also be linear Gaussian:

$$p(z_t|x_t) = \mathcal{N}(C_t x_t, Q_t)(x_t) \tag{17}$$

where C_t is the observation model which maps the true state space to the observed state space and Q_t is the covariance matrix describing the measurement noise. The generalized Kalman Filter can be seen in algorithm 3.

Algorithm 3: Kalman Filter

1 **function Kalman_filter** $(\mu_{t-1}, \Sigma_{t-1}, u_t, z_t)$**:**
2 $\quad \overline{\mu}_t = A_t \mu_{t-1} + B_t u_t$
3 $\quad \overline{\Sigma}_t = A_t \Sigma_{t-1} {A_t}^T + R_t$
4 $\quad K_t = \overline{\Sigma}_t {C_t}^T \left(C_t \overline{\Sigma}_t {C_t}^T + Q_t\right)^{-1}$
5 $\quad \mu_t = \overline{\mu}_t + K_t(z_t - C_t \overline{\mu}_t)$
6 $\quad \Sigma_t = (\mathbb{1} - K_t C_t)\overline{\Sigma}_t$
7 $\quad$ **return** $\mu_t, \sum_t$
8 **end function**

Figure 17: This algorithm is essentially the Bayes Filter algorithm for only Gaussian PDFs and linear measurement and control functions. It takes as an input the old mean μ_{t-1} and covariance Σ_{t-1} of the belief $bel(x_{t-1})$ as well as the control u_t and measurement z_t. In lines 2 and 3 the mean $\overline{\mu}_t$ and covariance $\overline{\Sigma}_t$ of the intermediate belief are calculated using the matrices A_t and B_t and the covariance R_t belonging to the action transition function. In the next line the matrix C_t and covariance Q_t of the measurement function are used to first calculate a matrix called the Kalman Gain K_t. This Gain is then used to calculate the new mean μ_t and covariance Σ_t in lines 5 and 6. [5]

The 2D case allows us to visualize the simplest scenario where the robot now moves along the x and y axes, and instead of sensing its distance to just the end of a track or a wall, it measures both is x and y coordinates. The behaviour is the same as in 1D but now colored contour regions denote the probabilities, as depicted in the thumbnail of the main menu (Figure 18). The settings are also the same as for the 1D case as seen in in figure 19. In Figure 20 we can see the big red area denoting the high initial uncertainty, which, after the measurement update, gets drastically narrower as shown in Figure 21. The robot then moves up and down for several time-steps without sensing, this results in the robot's *belief* drifting away from the actual location of the robot and also in a Gaussian that is flattened out in the y dimension, see Figure 22. Note that the uncertainty along the x dimension stays the same and is only the proportion $\frac{uncertainty\ in\ y\ dimension}{uncertainty\ in\ x\ dimension}$ that increases.

As we have seen, Kalman Filters have two main advantages over Histogram Filters, one being the memory and computational efficiency and the other being the ability to represent the *belief* in continuous spaces. However, there is one important limitation of Kalman Filters, they can not represent multi-modal distributions since the *belief* is always a normal distribution which is uni-modal by nature. The Particle Filter method discussed in the next section can both deal with continuous space and represent multi-modal distributions in the *belief*.

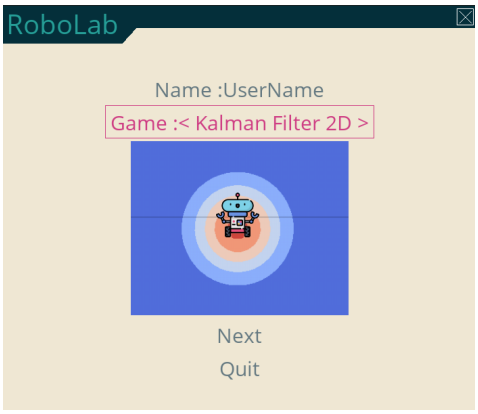


Figure 18: Kalman Filter 2D interface main menu.

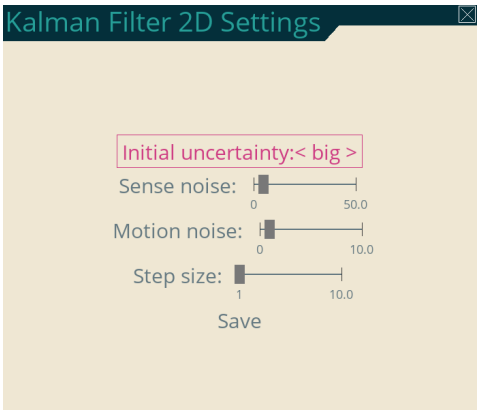


Figure 19: Kalman Filter 2D interface settings.

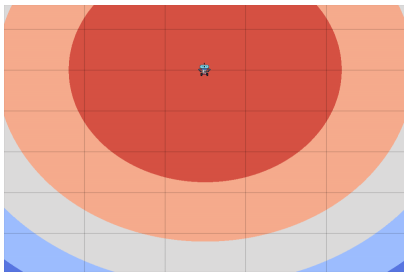


Figure 20: Kalman Filter 2D interface where the robot has not perform any motion or measurement yet. The very wide gaussian represents the high uncertainty at initialization.

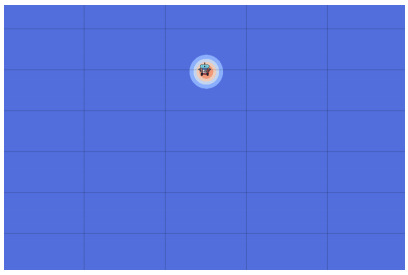


Figure 21: Kalman Filter 2D interface where the robot has just performed a measurement update. The very narrow gaussian represents the increased certainty due to integrating sensor information.

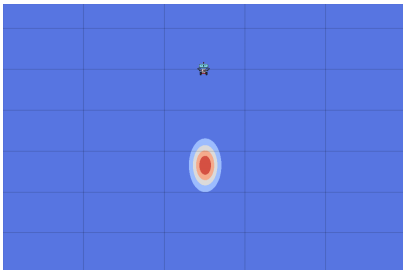


Figure 22: Kalman Filter 2D interface where the robot has performed several vertical motion steps without sensing the environment. As a consequence, the robot's estimate has drifted away from its actual location and the Gaussian has flattened out in the y axis.

3.2.6 Particle Filter

The Particle Filter is another implementation of the Bayes Filter 3. In order to represent the *belief* multiple "clones" (Particles) of the robot are created, each with a different state, for example different location and orientation, and each represents a hypothesis of the possible pose of the robot. These particles are distributed according to the *belief* and the average of all these hypotheses represents the best current state estimate.

The set of all M particles at time t will be denoted as: $\chi_t := {x_t}^{[1]}, ..., {x_t}^{[M]}$. This set is calculated using the previous belief/particles χ_{t-1}, the motion/control u_t and measurement z_t. First, in the motion update, each individual particle ${x_t}^{[m]}$ is sampled according to the transition probability $p(x_t|u_t, x_{t-1}^{[m]})$. Next, each particle receives a weight that is proportional to the measurement probability $p(z_t|{x_t}^{[m]})$, which represents how likely the observed measurement z_t is given the state of the particle ${x_t}^{[m]}$. Then M particles are resampled (with repetition) from the

set of weighted particles. Each particle $x_t^{[m]}$ is drawn with a probability proportional to its given weight. This results in the particles closest to the actual state of the robot getting resampled more times than the ones further away. The new set of particles will then be clustered tighter together around areas which are closer to the actual state.

The interface implemented to illustrate this algorithm consists of a simplified solar system representing the environment, and a rocket that can move forward or turn to the sides representing the robot. The state in this case consists of the location and orientation of the robot. Each particle has to visually show this information, a good candidate for this purpose are arrows, the direction of which denotes the orientation of the given particle. This can be seen in the thumbnail shown the *main menu* (Figure 23). The parameters that can be adjusted are shown in Figure 24.



Figure 23: Particle Filter interface main menu.

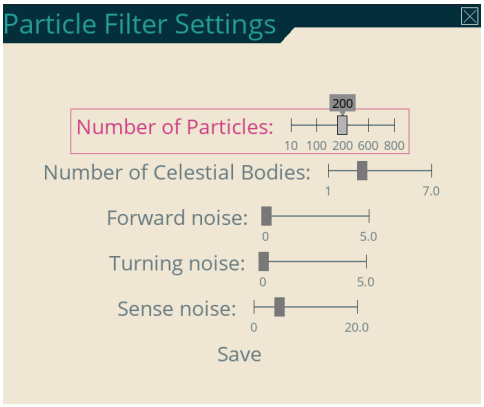


Figure 24: Particle Filter interface settings.

At initialization, M particles are spread out in space with coordinates x, y and orientation θ values according to the robot's uncertainty in each of these dimensions (this can be visualized in figure 25). Then when the robot gets a motion command, all the particles move according to the command but relative to their specific x, y and θ values. After this motion update there comes a measurement update where the robot senses its distance to each feature in space, this measurement z_t is used to assign weights to each particle depending on its position. To do this, each particle $x_t^{[m]}$ also measures its distance to the celestial bodies and uses this measurement $z_t^{[m]}$ as a mean value for a Gaussian which has a variance r proportional to the noise of the distance sensor. Then the probability density function (PDF) of this Gaussian is used to calculate how likely it is to get the value $z_t^{[m]}$ from a Gaussian with mean $z_t^{[m]}$ and variance r. This value is used as the weight for the given particle. Subsequently, M particles are then resampled with repetition according to their weights and the procedure repeats.

Similar to the Histogram Filter, the Particle Filter allows to represent multi-modal beliefs. In figure 26 we can see an example of this denoted by three clusters of particles. This happens because the particles in these areas are roughy the same distance away from the earth, moon and mars. Hence they will receive high weights and be resampled with higher probability. This ambiguity is generally found at the beginning of navigation. If the rocket keeps moving then the clusters that do not coincide with the actual location and orientation of the robot are going to move in other directions, thereby receiving smaller weights with each update [12]. Figure 27 shows the state a few

[12] In this interface motion and measurement are coupled so there is no need to manually perform the *sense* action since this is

time-steps after the one in Figure 26. The only cluster that survived is the one more consistent with the ground truth.

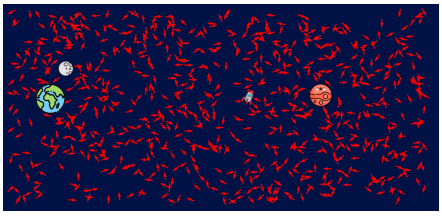


Figure 25: Particle Filter interface where the robot has not perform any motion or measurement yet. The very wide distribution of particles represents the high uncertainty at initialization.

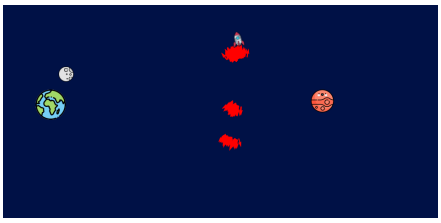


Figure 26: Particle interface where the robot has just started navigating. The three clusters represent the ambiguity of three different areas that are consistent with the robot measurements.

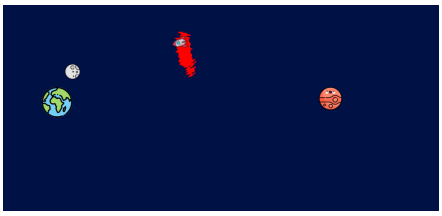


Figure 27: Particle Filter interface where the robot has navigated for several time-steps. As a consequence, the ambiguity has been resolved and the particles are converging to the actual location and orientation of the robot.

The noise parameters play a fundamental role in the performance of this algorithm. Too much noise and the particles will not converge, too little and the particles degenerate. Particle degeneration is when only a small subset of particles is resampled. This can happen if the sensor noise is too low because then the particles will receive a higher "penalty" (small weights) the less similar their measurements are to those of the robot. Motion noise is also crucial, to understand why lets suppose that motion is completely deterministic (i.e. noise is zero). Then each particle will move and rotate exactly the same amount every time but since the starting poses were different their paths will not "synchronize" with that of the robot. The three figures shown below (28, 29 and 30) illustrate the effects of changing the noise parameters to different values.

The main advantage of this approach is that it deals with only a finite set of state samples which are distributed in continuous space. Hence, any type of distribution can be approximated, in contrast to Kalman Filters which can only represent Gaussian distributions. A multi-modal *belief* might be for example necessary when performing localization in symmetrical environments. Consider a room with two identical doors located in different walls, where the robot sees only one door and has no more information. In this case, we want two areas with high probability in our *belief*, but if we are using Kalman Filters, we would be forced to either pick one of the two doors as the more likely and use that location as the mean, or to set the mean it the middle point between the two doors and increase the variance to cover both areas. The resolution with which Particle Filters can approximate any given distribution increases with the number of particles, but so does the memory requirement, more specifically, the space complexity is *exponential* in the number of dimensions. This is because for each extra dimension, more particles are needed to represent the uncertainty across that dimension. For example, when dealing with the 3 bins example described in section 2.2.3 we might need only 3 particles to represent the belief, but if we add a y axis, then for each of the previous bins we would want to add 3 more across the new dimension, ending up with 9 bins in a two-dimensional grid, hence requiring 9 particles instead of 3.

automatically done with each movement.

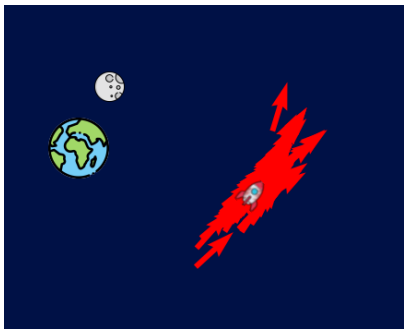


Figure 28: Particle Filter interface where the robot has high forward motion noise. The very wide spread of particles along the direction of motion is due to this noise.

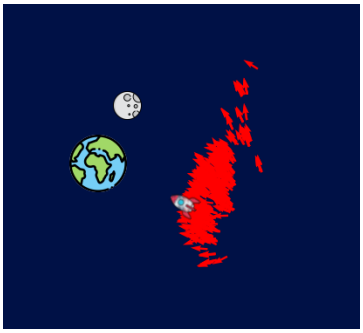


Figure 29: Particle Filter interface where the robot has high sensor noise. The "penalty" (i.e. small weights) to each particle is inversely proportional to the measurement noise. So with higher noise, particles that are further away from the robot still have a chance of being resampled. This creates the wide spread of particles around the robot.

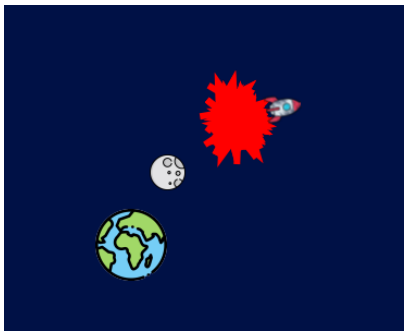


Figure 30: Particle Filter interface where the robot has high turning motion noise. With each turning motion there is high variation in the angle that each particle turned, which prevents the particles from converging to any particular orientations.

3.3 Motion Planning

In robotics, motion planning was originally concerned with problems such as how to move a piano from one room to another in a house without hitting anything. The field has grown, however, to include complications such as uncertainties, multiple bodies, and dynamics. In artificial intelligence, planning originally meant a search for a sequence of logical operators or actions that transform an initial world state into a desired goal state. Presently, planning extends beyond this to include many decision-theoretic ideas such as Markov decision processes, imperfect state information, and game-theoretic equilibria[13].

In the context of this work, robot motion planning is the problem of finding a path to a goal location given an initial location and a map of the environment. The following interfaces illustrate some of the different algorithms classically used to solve the planning problem.

3.3.1 A*

When trying to find the shortest path from a source node to a goal node in a graph, there are different search strategies that can be taken.

- ***Breadth first search*** (BFS) visits all nodes that are at distance d (number of edges) away from the source node before exploring nodes at a distance $d + 1$.
- ***Depth first search*** (DFS) uses a fixed order to select a neighbor node to explore next. Then, using this order, recursively keeps visiting the successor of the current node, until either the goal or a dead end is reached. In

the latter case, the algorithm goes back until the first node in the visited sequence which still has non-explored neighbors and repeats the process along that branch of the search tree.

- ***Cheapest-first search*** (CFS) is similar to BFS but, instead of using the *distance* as an ordering criterion, uses the *cost* from the source. So in this case the search three has weighted edges, and the cost so far (the sum of weights of traversed edges) to each visited node is stored, subsequently the node with the *cheapest* cost so far is visited next.

All the above mentioned approaches are guaranteed to find the optimal path to the goal. However, since they do not consider the *distance to the goal* but instead search according to a specific fixed pattern in the case of BFS and DFS, and according only to the *distance from the source* in the case of CFS, they explore inefficiently. It is worth noting that CFS at least is optimizing the search by not going to far in wrong directions, which can happen with DFS, and its better than BFS since the *cost* so far is a more informed metric than just the number of "jumps" taken. Nonetheless, CFS uses no information about the location of the goal, hence also can search in "wrong" directions. For this reason they are known as *blind* or *uninformed search algorithms*. For example consider a scenario where the goal node is relatively close to the source. In this case if we also considered the distance to the goal when searching we could go directly to the goal instead of wasting time on nodes that are further away from the target. Algorithms that use information of the location of the goal when searching are known as *directed* or *informed search algorithm*.

- ***Greedy best first search*** (GBF) *only* considers the estimated distance to the goal when exploring nodes. If there are no obstacles this approach will go directly to the goal without unnecessary exploration. But, since the heuristic used is only an *estimation*, it might not consider obstacles that block the otherwise "direct" path to the target. Consider a scenario where the start node is at the beginning of a very long blind corridor and the goal is at the end of the corridor but on the "other side of the wall". Lets assume that there is also a door right at the beginning that goes out of the corridor and then there are no more obstacles so the goal is indeed reachable if we go through this door first. If we use *greedy best first*, we will explore greedily at each step and hence go down the complete corridor just to find out that it is a dead end. Then we would have to turn back to eventually go through the door and move towards the goal. This a typical scenario where a *blind* approach like CFS finds the best path faster than an *informed* one like GBF.

A-star (or A*) is basically the combination of CFS and GBF search. Here we use both the *cost so far* and the *distance to the goal* when assigning priority weights to the search queue. More precisely, a function $f(n) = g(n)+h(n)$ is used to assign these weights. Where $n \in N$ is a node, $g : N \to \mathbb{R}$ is the cost function and $h : N \to \mathbb{R}$ the heuristic function. Therefore, $g(n)$ is the *cost from the source to node n* and $h(n)$ is the *estimated cost to the goal from node n*. A* is also a type of *directed search*. But, by considering also the cost so far, it avoids inefficient exploration like the one depicted in the corridor example. A* is guaranteed to find the optimal path as long as the heuristic function h returns an *underestimation* of the actual cost to the goal. When this condition applies, h is said to be an *admissible* heuristic.

The interface described in this section implements A* search in a 2D grid world and allows the user to:

- Set the initial position.

- Set the goal position.
- Add or delete obstacles.
- Set the obstacle probability p for each cell and initialize a random grid according to p.
- Save and load a custom map.
- Set the grid width and height.
- Select the way the path should be shown, either by coloring the cells green or drawing arrows pointing in the next direction to take.

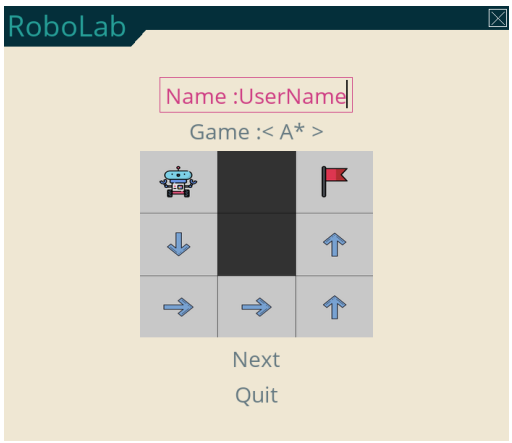


Figure 31: A* interface main menu.

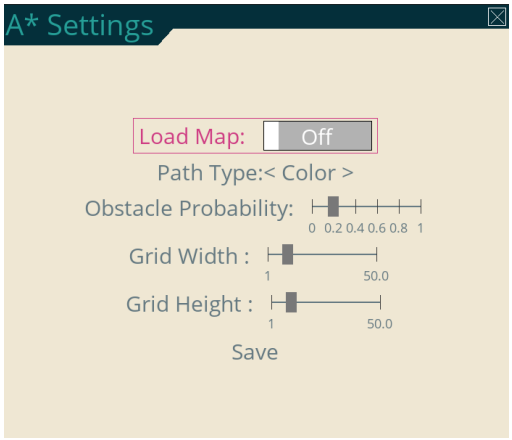


Figure 32: A* interface settings.

Once the user initializes the environment, a grid according to the parameters shown in figure 32 will be generated and rendered. Then with the left and right mouse buttons the source and goal can be set. Once both source and goal are given, the optimal path will be shown in case it exists. This can be seen in figures 33 and 34. Otherwise a path to the closest possible cell is shown with an exclamation mark at the end of the path to denote that the goal has not been reached, as illustrated in figure 35. The user can also use the center mouse button to create or delete obstacles in a given map. With every new obstacle added or deleted a new search is performed and the path is updated immediately. This is how the grid with the isolated target shown in figure 35 was created from the grid used in figures 33 and 34.

The motivation for allowing the user to select two different types of path representations was based on the fact that for very big grids the arrows are barely visible. In this case showing the path with color makes for a cleaner path visualization. This can be observed below in figures 36 and 37

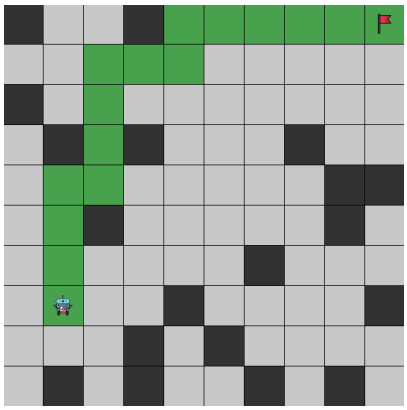


Figure 33: A* interface where the path is shown using colored cells.

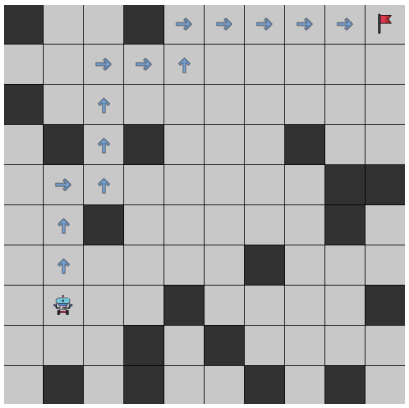


Figure 34: A* interface where the path is shown using arrows.

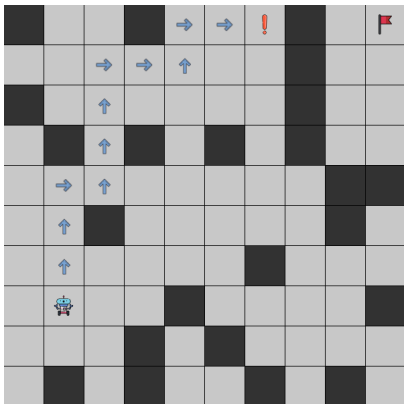


Figure 35: A* interface where there is no path to the goal. In this case an exclamation mark is shown on the closest cell to the goal.

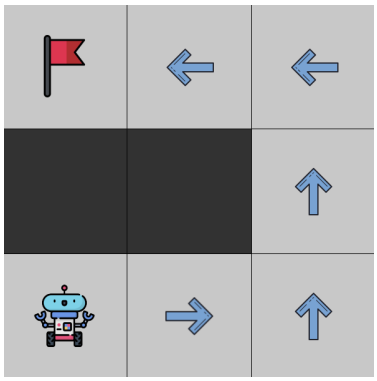


Figure 36: A* interface using a small grid world and arrows to represent the path.

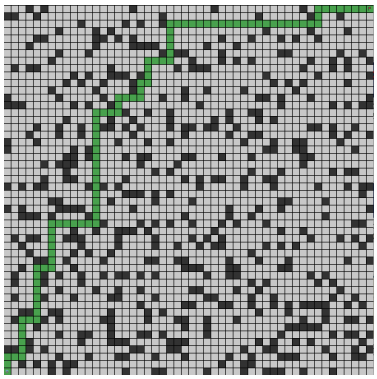


Figure 37: A* interface with a big 50x50 grid world using colors to represent the path.

3.3.2 Dynamic Programming

Dynamic Programming (DP) is a method that can be used to calculate the optimal path to a given goal from every single position in a map. The basic idea for a 2D grid is to divide the process in two steps:

1. Assign to each cell c_{xy} a value $f(x, y)$ which the distance from that cell to the goal. This is calculated recursively by assigning the value of 1 to the cells that neighbor the goal, then the value of 2 to the neighbors of these cells and so on. More precisely, each cell c_{xy} gets the value $f(x, y) = \min_{x',y'} f(x', y') + 1$. Where $c_{x'y'} \in$ *Neigbohrhood of* c_{xy}. So $\min_{x',y'} f(x', y')$ is the value of the optimal neighbor of c_{xy}. That is, the distance to the goal from the neighbor that is closest to the goal. Intuitively, for a cell c_{xy}, its distance to the goal $f(x, y)$ is equal to the distance from the closest neighbor to the goal $f(x', y')$ plus one more step, which is the step necessary to reach that neighbor.

2. Assign to each cell the action needed to move to the optimal neighbor. In a 4-connected grid world the possible actions are *up, down, left* and *right*. Therefore, if we represent this with arrows, each cell in the grid will be assigned an arrow pointing to the neighbor closest to the goal. This is known as finding a *policy*, that is, a mapping from states to actions.

Since following the arrows leads to the goal in the most optimal way, this is equivalent to finding the optimal path for every location in the map. To understand why this would be useful, consider the case where a robot is using A-star for path planning, and some event causes it to deviate considerably from its original plan. Then it would need to *replan* again to find a path to the goal from the current position. In fact, every time the robot is put in different initial locations with the same goal in this environment it would need to replan (i.e. running A* again). In contrast to this, by running the Dynamic Programming algorithm mentioned above only *once*, there would be no further replans necessary for a given goal in the given environment. Hence, the robot could immediately start moving towards the goal regardless of its initial position by using the calculated policy.

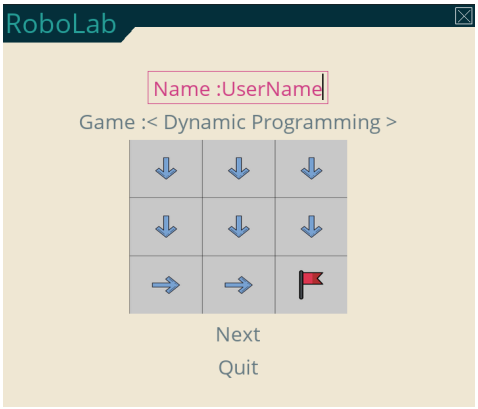


Figure 38: Dynamic Programming interface main menu.

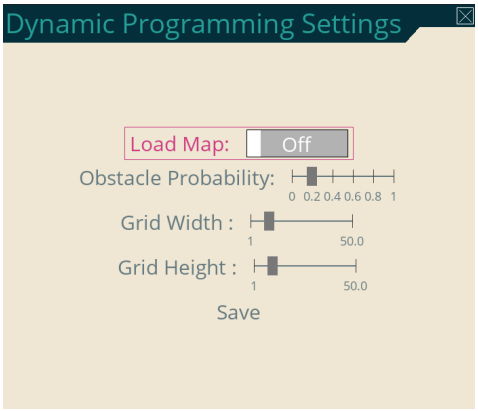


Figure 39: Dynamic Programming interface settings.

This interface works very similarly to the one shown in the A* section. The goal can be set with the right mouse button and the arrows will be immediately updated. The main differences are that that there is no start position to be set and the policy (i.e. the paths) is only shown as arrows. In figure 40 random initial state in a 6x6 grid is shown. In this case, if the goal is not reachable from some area of the map, then this is depicted by the simple absence of arrows in that region, denoting that there is no path to the goal from there. This can be seen in figure 41.

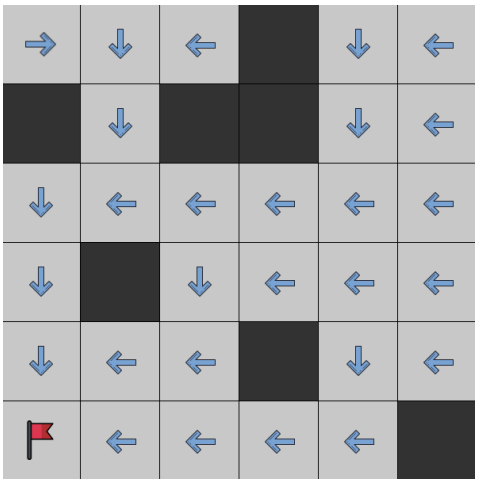


Figure 40: Dynamic Programming interface with a random initial state.

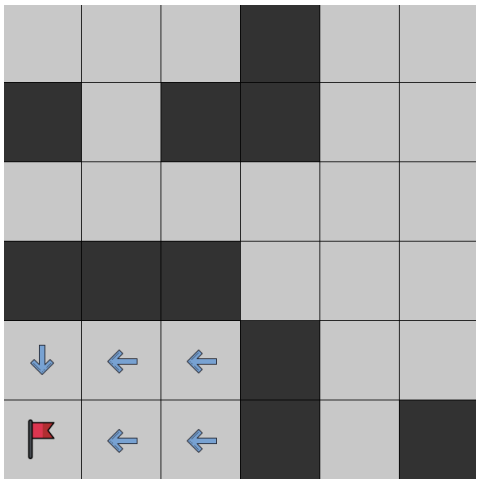


Figure 41: Dynamic Programming interface with a goal that can only be reached from a small area of the map. There are no arrows drawn in the regions where there is no path to the goal.

Although DP has some advantages as was explained previously, it also has the main disadvantage that it is much more computationally expensive than A*. For this reason it might be impractical to use DP in very big environments.

3.3.3 Optimum Policy

Dynamic Programming is actually a general method for finding optimal policies in Markov Decision Processes (MDP) where the rewards and the transition probabilities are known. In the previous section we considered the specific case where all the actions where just move actions from one cell to the other and all had the same cost of 1. More generally, there are cases where taking some actions can have a higher cost than others and where there might be other actions besides just moving (e.g. a warehouse robot that can perform *pickup* or *drop* actions).
To understand why non-uniform action costs can be desirable, lets consider an example. Suppose that the robot in question is an autonomous taxi transporting passengers around a crowded city. Lets say that there are two main optimization criteria when planning a path to a destination: speed and safety. On the one hand we want the shortest path in order to transport the client faster to the destination, on the other hand we want to take the safest path so the client arrives unharmed. Moreover, lets say that we know that left turns are trickier and hence riskier to take because of the traffic on the parallel street being crossed. Therefore, in order to disincentivize left turns, one would want to assign them a higher cost than the safer options of right turns or forward moves. This principle can be generalized to penalize any action or also use different penalties depending on the current state.

The interface implemented was inspired by the simple example above. It consists of a grid world with a car that needs to find the best plan (i.e. the best policy), which is found considering the cost of each different action, to get to the goal given by the user (see thumbnail in figure 42). The underlying algorithm to find the best policy is the

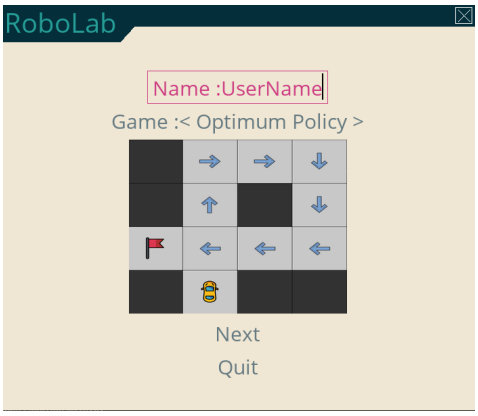


Figure 42: Optimum Policy interface main menu.

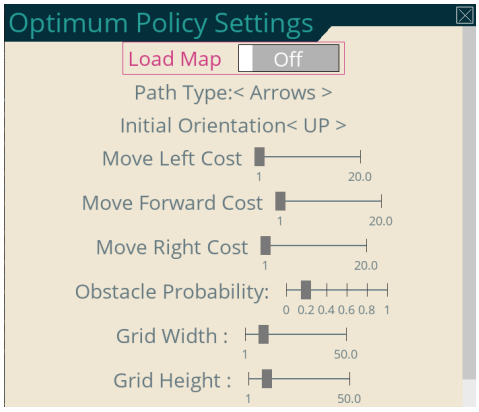


Figure 43: Optimum Policy interface settings.

same as the one described in the DP section. The only difference is that now we are considering only three possible actions (*left, forward*, and *right*) that depend on the current orientation of the robot. This means we have to add the extra dimension of *orientation*. Before, only x and y coordinates were enough to find the neighbors of any given cell c_{xy}. Now x, y and θ are needed, where $\theta \in \{up,\ right,\ down,\ left\}$ represents the current orientation considered from a fixed point of reference (as seen from above [13]). Note that now the neighborhood is composed of maximum 3 cells instead of 4, this is because our taxi cannot move backwards. The complete algorithm behaves as combination of A* and DP in the sense that it finds a policy for every starting position (DP) but receives as input a specific initial position and orientation and shows the optimal path for that specific initial state (A*). Accordingly, the process can be separated in two steps:

1. Iteratively find the optimal neighbor for each cell and update the value of the current cell to *value of optimal neighbor* + *cost of action to reach optimal neighbor*. This is done until there is no further change to the value of any cell, meaning that we have found the optimum policy $\pi : S \rightarrow A$ where S is the set of possible states which, in turn, are each composed of a tuple $s = (x, y, \theta) \in S$, and A is the set of possible actions. For example in a 1x2 grid with the goal at $(0, 1)$ and the initial state $s_{init} = (0, 0, right)$, then it would be the case that $\pi(s_{init}) = forward$.

2. Starting from the initial state s_{init}, draw an arrow corresponding to the policy for that state, e.g. if $\theta = left$ and $\pi(s_{init}) = right$ then the drawn arrow for that cell would be $\uparrow$ [14]. Then, move one cell in the direction of the arrow and repeat the process until the goal is reached.

The resulting behaviour is similar to the one seen in the A* interface. That is, once one has selected a start position and a goal, a path will be drawn either with arrows or colors. The difference in this case is that since actions can have different costs, the optimal path is no longer simply the spatially closest route. Now there can be loops in the optimal path, for example when trying to avoid a left turn results in three right turns instead (see figure 45).

Although the "risky left turn" example was used as a motivation for the interface, the user can set the cost of each

[13] In this case the fixed frame of reference is the Cartesian coordinate system

[14] The reason is that, when observed from above (fixed frame of reference), if the robot is facing to the left and then takes a right turn, it would then be facing upwards.

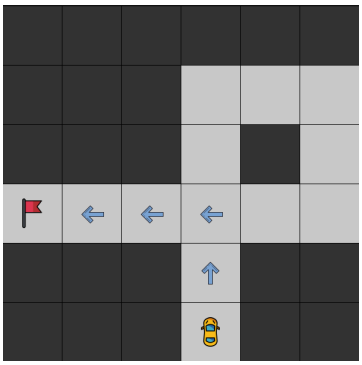


Figure 44: Optimum Policy interface where the action cost is uniform across all actions. Note that the resulting behaviour is the same as in the A* interface.

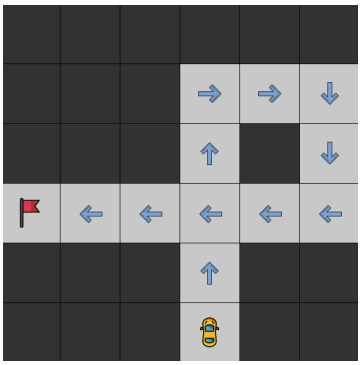


Figure 45: Optimum Policy interface where the action cost is not uniform. Namely $left = 15, forward = 1, right = 1$. Note how the optimal path in this case involves three right turns and multiple extra forward moves in comparison to figure 44.

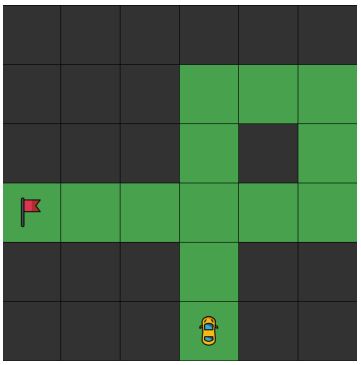


Figure 46: Optimum Policy interface where the action cost is not uniform. The action costs are the same as in figure 45 but here the path is represented with colored cells instead of arrows.

action (as seen in the settings menu shown in figure 43). For example one could set the cost of the right turn to be much higher than the other action, this would result in an opposite but symmetrical behaviour as can be seen in figure 47

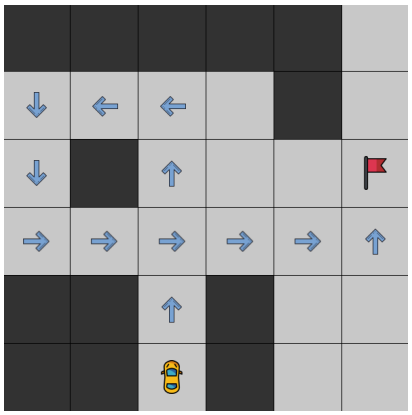


Figure 47: Optimum Policy interface where the action cost is not uniform. In this case $left = 1, forward = 1, right = 15$.

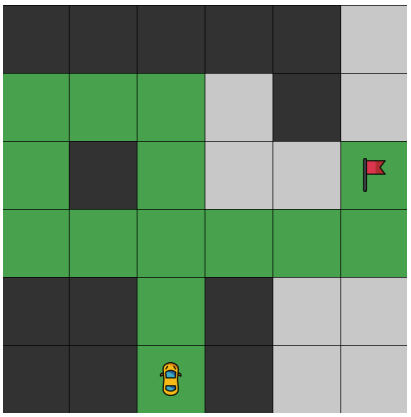


Figure 48: Optimum Policy interface where the action cost is not uniform. The action costs are the same as in figure 47 but here the path is represented with colored cells instead of arrows.

3.4 Smoothing & Control

After localizing the robot and calculating the best path to the goal, there is still work to be done for the robot to reach the desired goal. First, there is the need to convert the discrete plan, which is based on a simplified representation of the world (in this case a grid), to a continuous set of coordinates for operation in the actual world. Once we have this continuous path the robot still needs to execute the actual motion commands in order to move along the target trajectory. The first step is called *Path Smoothing* and the second is called *Control*. Both of these procedures will be explained in detail in this section.

3.4.1 Path Smoothing

The path returned by A* or DP is, as we have seen in the previous interfaces, a discrete set of coordinates on the grid where each specify the next location the robot should move to, the last of which is the actual goal. This path has three main limitations:

- It assumes the robot can only move *up, down, left or right* and not in any other direction in between. It is easy to see why this is far from optimal. Consider a world without obstacles where the initial position is on the lower left corner and the goal on the upper right. The discrete plan returned by the versions of A* and DP presented here would tell the robot to go all the way up and then right or all the way right and then up, instead of taking the actual shortest path which is taking the main diagonal that goes across the grid.
- It assumes the robot can make hard 90 degree turns on the spot [15]. In the same example used above, after the robot reaches the upper left corner it would have to turn 90 degrees to the right to then be able to keep moving forward to the goal. Note that normal cars cannot perform such turns.
- It ignores the space in between coordinates. Actually any path planning algorithm *necessarily* has to perform some type of discretization of the world since there is no such thing as contiguous coordinates in continuous space. However, depending on the coarseness of the world representation in which planning was done, we might want to perform interpolation to create intermediate coordinates between points in the original path. Consider the case of a 2x2 grid world representing a room in which we want the robot to follow a roughly circular path, this would be impossible with just one coordinate per grid cell.

There are two main criteria to consider when smoothing a given discrete path: *smoothness* and *similarity to the original path*. Therefore we need to simultaneously optimize for both of these properties which are opposite to each other. Consider the example illustrated in figures 49 and 50, the *smoothest* path is the red diagonal line from the initial position (lower left corner) to the goal (upper right corner). The path *most similar to the original path* is the green line, which is just a copy of the original path. In order to optimize for *smoothness* we want that each point p'_i in the new path is close to its neighbors p'_{i-1} and p'_{i+1}, or in other words, we want p'_i to be close to $\frac{1}{2}(p'_{i-1} + p'_{i+1})$ which is the point exactly between the neighbors. This is clearly not the case for the green path in the example, the upper left point would be the closest to both its neighbors if it was exactly in the middle point of the red diagonal. In order to optimize for *similarity to the original path* however, we also want that each point on the new path is

[15] A robot capable *holonomic motion* would not need to turn since it can also move laterally. But this is just another assumption.

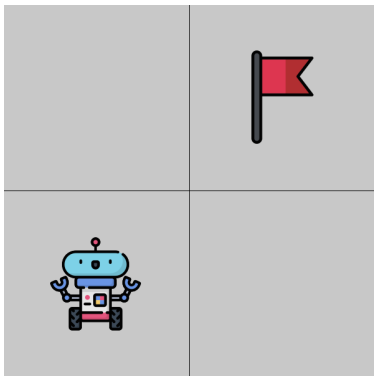


Figure 49: Simple 2x2 world with initial position and goal on opposite corners.

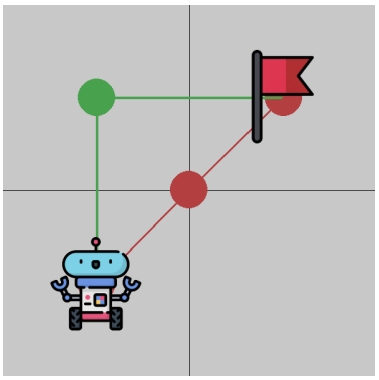


Figure 50: There is the discrete path on green and the smoothest possible path on red.

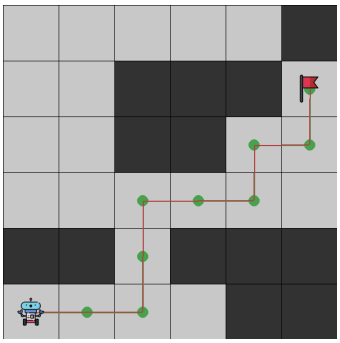


Figure 51: Path Smoothing interface where only the *similarity to the original path* criterion was optimized (i.e. $\beta = 0$).

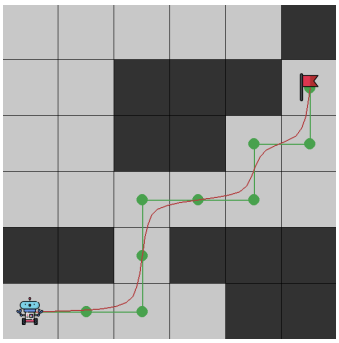


Figure 52: Path Smoothing interface where both the *similarity to the original path* and *smoothness* criteria were optimized (i.e. $\beta \approx \alpha$).

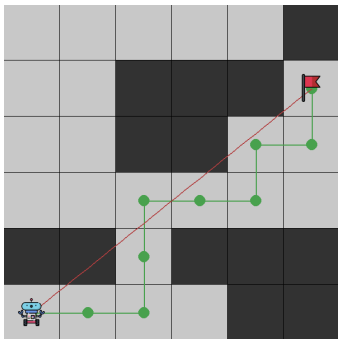


Figure 53: Path Smoothing interface where only the *smoothness* criterion was optimized (i.e. $\alpha = 0$).

close to the corresponding point in the original path, this is not the case for the red path in the example, the middle red point would be closest to its corresponding point in the original path if it was exactly where the green upper left point is. If p_i are the points on the original path and p'_i the ones on the smooth path, we want that the point p'_i is close to both p_i and also to $\frac{1}{2}(p'_{i-1} + p'_{i+1})$. Moreover, depending on the application, one or the other criterion could be more desirable, therefore we want to be able to control the trade-off between these two properties, that is, moving each p'_i more in the direction of the smoothest point or more in the direction of the corresponding p_i point. This can be achieved by the path smoothing algorithm shown in figure 54. Note that this is equivalent to performing gradient descent with the loss function $\sum_{i=1}^{n-1}(p_i - p'_i)^2 + (\frac{1}{2}(p'_{i-1} + p'_{i+1}) - p'_i)^2$.

The interface is similar to the ones presented in the planning section. The main difference is that now each point in the original path is drawn as a green circle with lines of the same color interconnecting neighbouring points. The *smooth path* is also drawn but in red and without emphasizing the points. The user can control the *smoothness* of the path (i.e. the ratio $\frac{\alpha}{\beta}$) by moving the mouse wheel up or down. Figures 51, 52 and 53 show the same path with

Algorithm 4: Path Smoothing

function Path_Smoothing $(path, \alpha, \beta, \tau)$**:**
  $smooth_path \leftarrow path$
  $done \leftarrow false$
  while *not done* **do**
    $total_change \leftarrow 0$
    for *i = 1 to path_length -1* **do**
      $old_value \leftarrow p'_i$
      $p'_i \leftarrow p'_i + \alpha(p_i - p'_i) + \beta(\frac{1}{2}(p'_{i-1} + p'_{i+1}) - p'_i)$
      $total_change \leftarrow |old_value - p'_i|$
    end
    if $total_change < \tau$ **then**
      $done \leftarrow true$
  end
  return *smooth_path*

Figure 54: Path Smoothing algorithm where the smooth path is initialized as a copy of the original path and then each point is moved a small step in the direction of both the corresponding original point and the point between the neighbors. The first and last points in the path are fixed, hence excluded from the optimization, otherwise the *smoothest path* would collapse into a single point by decreasing the distance between all neighbors. α is the parameter that controls the *similarity to the original path* and β controls the *smoothness*. This is done iteratively until the total change after one iteration is below a tolerance parameter τ. The proportion $\frac{\alpha}{\beta}$ controls the overall shape of the smooth path. Where $\lim_{\frac{\alpha}{\beta} \to 0}$ *new path* = *smoothest path* and $\lim_{\frac{\alpha}{\beta} \to \infty}$ *new path* = *original path*.

no smoothing, a good amount of smoothing and too much smoothing respectively. Note that in the extreme case shown in figure 53, the smoothed path does not respect the obstacles.

Interpolation between contiguous points was done in order to obtain a higher resolution path. However, the time complexity of the smoothing algorithm is exponential in $n * |path|$, where n is the number of interpolated points between two contiguous points and $|path|$ is the length of the original path, which on average depends on the size of the world denoted as n_cells. Because of this, in order to decrease the smoothing time and make the interface more responsive, different values for n where used depending on the size of the grid with the goal of limiting the value of the product $n * n_cells$. Namely, $n_cells < 20 \rightarrow n = 8$, $20 \leq n_cells < 40 \rightarrow n = 4$, $40 \leq n_cells \rightarrow n = 2$.

3.4.2 PID Controller

Once a way to continuously estimate the robot's position and a continuous target trajectory is available, the robot needs to be able to *send specific commands to its actuators* (e.g. wheels, rotors or legs) to follow the desired path. This is known as *The Control Problem*, and one of the most widely used and reliable methods to tackle it are *PID Controllers*.

At every time step t there will be some distance d between the estimated robot position p'_t and the target position p_t, that is, the closest position to p'_t that is exactly inside the desired trajectory. This distance is the *error* for that time step. The first thing that the robot wants to do is get closer to the desired path, thereby reducing the

error. The speed, or the strength of the steering, with which it should move towards the optimal position should be *proportional* to the current distance d. In other words, the further away it is from where it should be, the faster it will move towards its target position. Algorithms that try to reduce the error only by considering this distance are hence known as **P Controllers**, where the **P** stands for *proportional*. The term that controls this behaviour is called the *proportional term*.

However, since the robot is continuously moving towards the desired position up until the moment when it reaches it, its inertia will cause it to *overshoot* and start getting away from its desired position again. This process repeats, causing an oscillatory behaviour. In order to avoid this overshooting, we would want to slow down our speed, or decrease the magnitude of the steering, as we get closer to the desired position, and this slowdown should be proportional to the speed with which the robot is getting closer to the goal position. Put differently, we want to *counter-steer* proportionally to the *difference* of the distance to the goal over time. This "difference in distance over time" is another way of saying "the speed with which the robot gets closer to the goal position". By adding this counter-steering to our P controller we would have a **PD Controller**, where the **D** stands for *differential*. Now, as the robot gets closer to the target position with a speed *proportional* to the error, it will simultaneously counter-steer with a strength proportional to that speed. As it gets closer to the goal, the proportional steering and differential counter-steering will balance out and the oscillatory behaviour will be no more. The *differential term* is the parameter used to regulate this counter-steering.

If all the sensors and hardware of the robot are *perfectly* calibrated, the PD controller is enough to bring the error down to zero. On the other hand, the existence of any type of deviation of measured values from actual values in the system, will cause the robot to consistently have an error between its estimation and the actual current state. This is known as *systematic bias*. In the case of an autonomous car trying to follow a straight path forward, the proportional and differential terms should allow it to smoothly follow the path. However, if for example the sensor that measures the steering angle is off by a few degrees, then the robot will constantly drift away from the desired trajectory. This drifting is counteracted to some degree by the PD controller, so the robot stays close to the desired trajectory. But, since the steering commands are based on the current steering angle estimation, and this estimation is wrong, there will always be a small error between the actual trajectory and the target one. The actual trajectory will then draw a line right next to the desired trajectory. The space between these two lines form an area that grows continuously over time. The bigger the offset in the sensor, the bigger the error and the area will be. In order to correct this error, we want to *counteract the counter-steering* by an amount *proportional to the area* formed between actual and desired trajectories. From calculus we know that to calculate the area between a curve $f(x)$ and the x axis in the interval $[a, b]$, we need to *integrate* $f(x)$ w.r.t. x over that interval, i.e. $\int_a^b f(x)dx$. For this reason the parameter used to control the weight given to this "counter-counter-steering" based on the area of the error is called the *integral term*. A controller that uses this and the other terms is hence called a **PID controller**, where the **I** stands for *integral*.

The interface implemented for this algorithm is different than all the others presented so far. Until now all simulation environments were static in the sense that the rendered graphics would only change after receiving user input. In this interface, an autonomous car trying to stay on the center of a highway is used to illustrate the behaviour of PID controllers. The perspective is a bird's-eye view centered on the car as seen in the thumbnail in figure 55.

The highway represents the background and it is constantly moving at a speed equal to that of the car but in the opposite direction to create the visual effect of the car's motion. On the upper left corner there are multiple values displayed, from top to bottom:

1. **CTE** stands for *cross track error* and is in this case just the distance from the car to the center of the highway.
2. **P** stands for *proportional* and is the proportional term of the controller.
3. **I** stands for *integral* and is the integral term of the controller.
4. **D** stads for *differential* and is the differential term of the controller.
5. **Drift**: If the steering wheel is perfectly *aligned* and *centered*, the steering angle should be zero, that is the angle that the front wheels form with the longitudinal axis of the car. The amount by which the steering angle deviates from zero, given that the steering wheel is perfectly centered, is called drift. Therefore drift is a measure of the *unaligment* of the steering wheel which is in turn a form of *systematic bias*. The drift can be positive or negative depending on the direction of the wheel's deviation, i.e. left or right instead of perfectly forward.
6. **Speed**: The magnitude of forward velocity of the car.

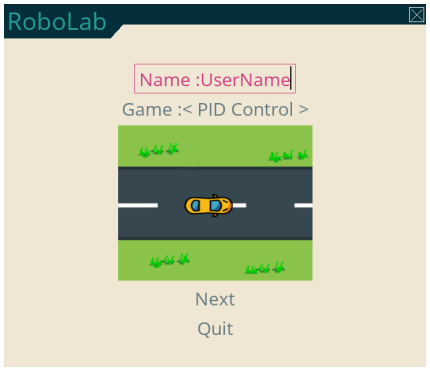


Figure 55: PID-Control interface's main menu.

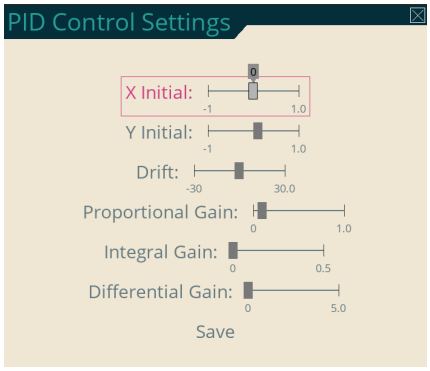


Figure 56: PID-Control interface's settings.

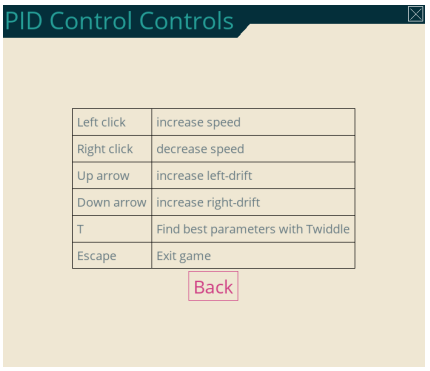


Figure 57: PID-Control interface's controls.

In the *Settings Menu* (figure 56) the user can set the starting position of the car by controlling the *X Initial* and *Y Initial* values, where for example $X = -1$ represents the left and $X = 1$ the right border of the window. Analogously for *Y initial* and upper/lower borders. The *PID terms* and *initial drift* can also be manually tuned on this menu. Figure 57 shows the commands the user can use while the interface is running. Particularly interesting is the command *"Find best parameters with Twiddle"* which gets executed when pressing the **T** key. *Twiddle* is a *stochastic hill climbing* algorithm used to optimize a set of parameters given a score function. In this case the parameters are the PID terms and the score function is the accumulated CTE over a fixed number of time-steps. The algorithm works by increasing or decreasing each parameter by an initial amount (probing values) and then using

the score function to asses the performance, if it is better, then the new parameters are kept and the corresponding probing value is decreased. If on the other hand performance is worse, then the probing direction is changed (i.e. if the values are *increased* or *decreased*) and again performance is assessed, keeping the values if it improved. The procedure is then repeated until the sum of the probing values is lower than a given *tolerance*, meaning the algorithm is converging to a local minimum.

Because of the dynamic nature of this environment its behaviour is best visualized by video or real-time interaction. However, a sequence of images can also be used to illustrate its main features. Accordingly, the next set of figures 58, 59 and 60 each show a sequence of screenshots to illustrate different behaviours depending on the values of the PID terms. A detailed explanation is presented in each case on the figure descriptions.

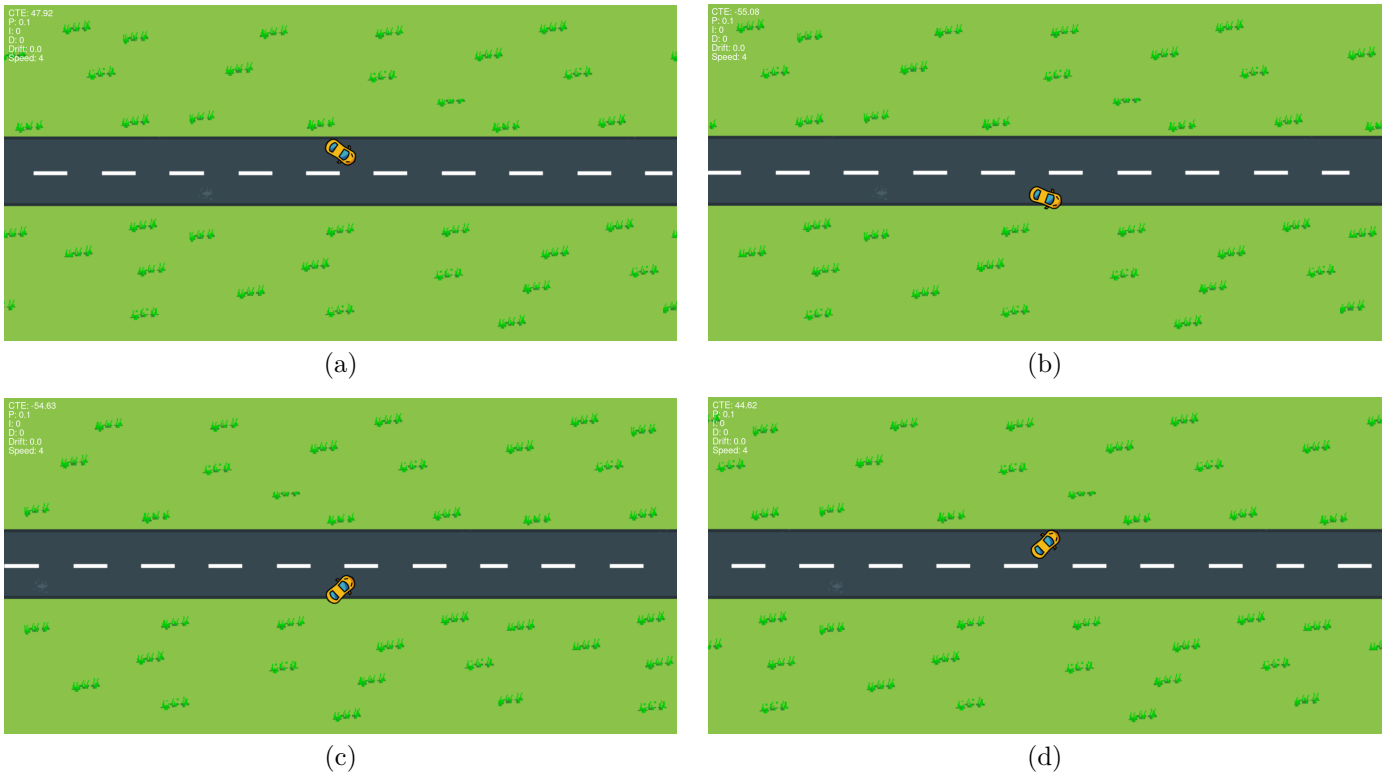


Figure 58: PID controller interface where the differential and integral terms are zero, in effect making it a P controller. By looking at the images in order, that is (a) → (b) → (c) → (d), one can see the oscillatory behaviour characteristic of P controllers. In (a) the car is moving towards the center of the highway but overshoots as can be seen in (b). The car then again tries to correct the error by steering towards the center but overshoots again as depicted in images (c) and (d).

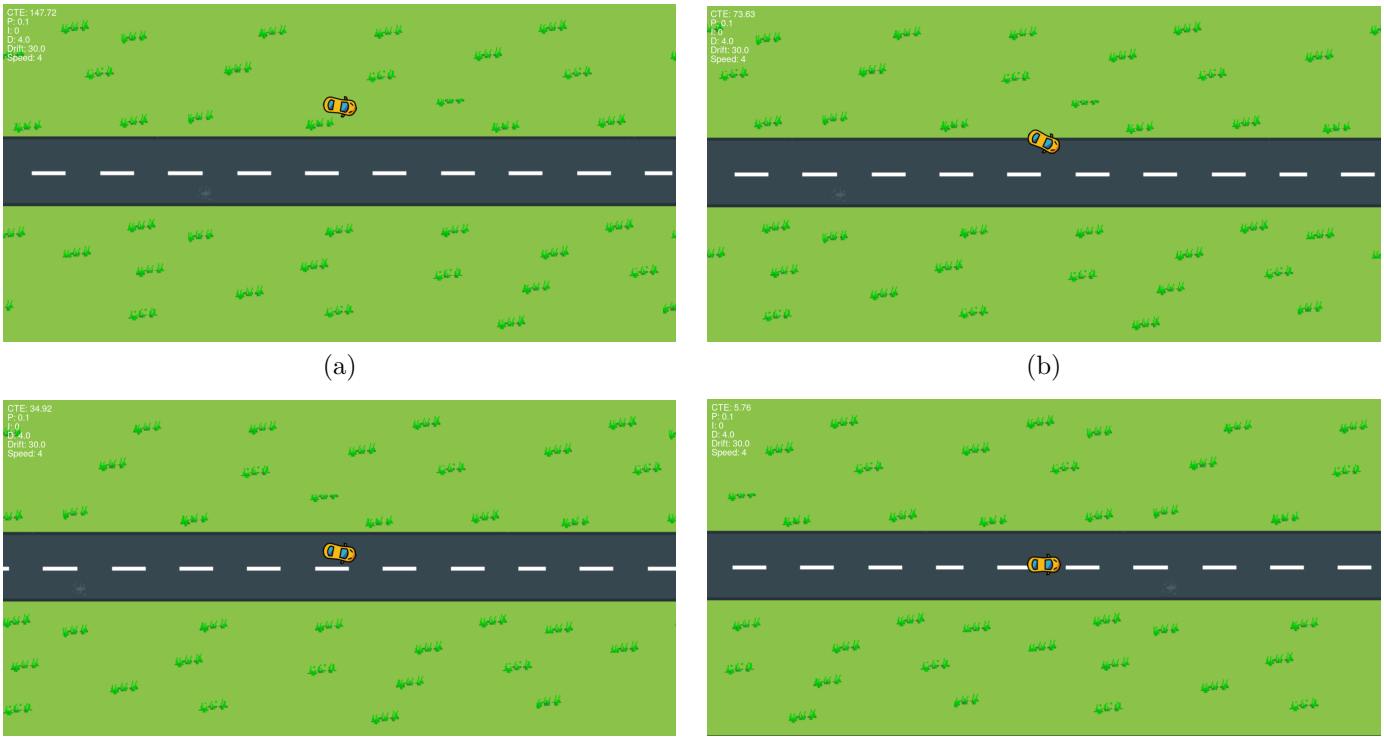
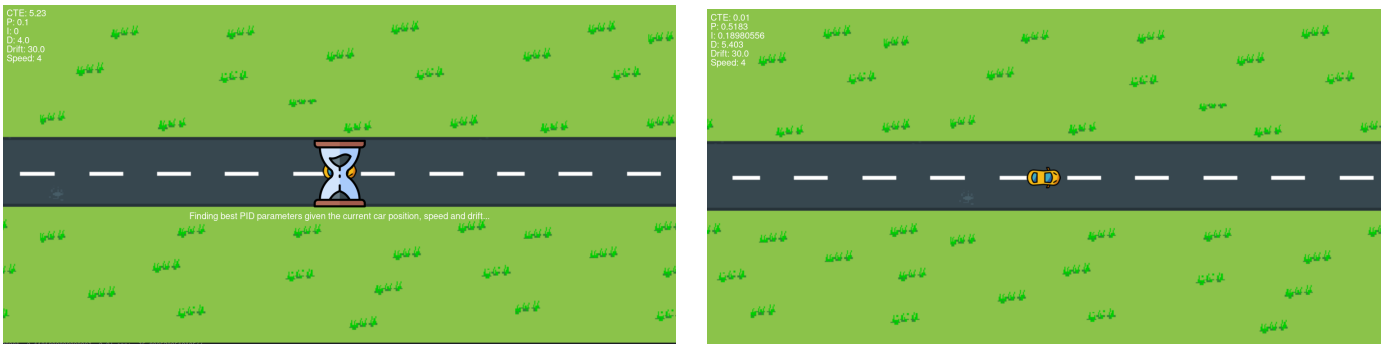


Figure 59: PID controller interface where only the integral term is zero, in effect making it a PD controller. By looking at the images in order, that is (a) → (b) → (c) → (d), one can see the that, in contrast to the behaviour shown in figure 58, the car smoothly approximates its target trajectory at the center of the highway. It is worth noting how, after stabilizing, the car follows a trajectory that is slightly above the center line of the highway, hence the error is not brought down all the way to zero as can be seen in the upper left corner of image (d), where $CTE = 5.76$. The reason why the PD controller cannot bring the CTE to zero is because of the high *systematic bias*, in this case denoted as *drift*=30.



(a) (b)

Figure 60: PID controller interface where none of the terms is zero. The first image (a) is a continuation of the state (d) in figure 59. Here the user pressed the **T** key to execute the *Twiddle* algorithm to search for the best parameters given the current state of the system. Image (b) shows on the upper left the values of the parameters found by Twiddle, note the non-zero integral term. After adding a small integral term the *cross track error* can be brought down to zero even with a high drift of 30 degrees. When one carefully compares (d) from figure 59 and (b) from this figure, it can be noticed that (d) is a little bit off from the desired trajectory, namely it is above the exact center of the highway. In contrast, the car in (b) is perfectly centered in the desired trajectory. This can be confirmed by comparing the CTEs of 5.76 vs 0.01 in (d) 59 and (b) respectively.

.

The specific example presented here is just a simple case that allows to illustrate the most important principles of PID Control. However, the concept is very general and can be used in multiple other systems, for example PID Controllers are widely in complex systems that need to adjust pressure or temperature [14].

4 Discussion

Each section presented in this thesis had as its main focus to show particular features that are very easy to understand when presented in a visual, interactive manner but otherwise relatively hard to comprehend just by analyzing formulas or code. A great example of this is seen in the Particle Filter interface, specifically in the changes in behaviour and distribution of particle clusters given different noise values. Using the interface it is immediately apparent how the noise affects particle convergence or degeneration, in contrast to analyzing static plots or run-time console outputs. Another good example is the PID controller interface where one can directly see the oscillatory behaviour of P controllers, how this oscillation can be fixed by adding a differential term, and observe the robustness of a given set of parameters by changing the speed or drift at run-time.

As mentioned at the beginning of this thesis, specific implementation details were ignored. However, several challenges appeared during the development process. Specifically, most of the implementation time was dedicated to rendering the graphics efficiently and making design decisions as opposed to developing and tuning the specific algorithms. For instance, most scientific plotting libraries were not originally designed to continuously render updates in real-time environments but rather to analyze static sets of data. On the other hand, most game development libraries, which are indeed designed with rendering efficiency in mind, do not have tools to plot mathematical

functions. As a consequence, one of the main optimization objectives was to be able to integrate plotting libraries with game development tools as seamlessly as possible. The main metric for this was the *frames per second* (fps), which was improved across multiple iterations. For example, the 2D-Kalman-Filter interface started at around 2-5 fps, which after restructuring the code to reuse data, improved to 10-15 fps. Then it improved to 25-30 fps after switching from moving and converting images around to directly using the figure rendering back-end of the plotting library to more efficiently integrate the plotting output into the interface. Even after these improvements, the size of the main window had to be constrained to be able to offer a smooth experience. An example of another feature that evolved iteratively was the settings and controls interface. Initially it was a lateral section visible at all times while running the specific environment. This, however, affected the aesthetics of the application, so it was then moved to a separated menu that was only visible once clicked upon. While this was better, it affected the user experience since it was combining two windows with different purposes into one, which was sub-optimal. Therefore the final iteration modularized the application into a hierarchical system of menus to select and setup the interfaces before running them. It allowed for a cleaner and more intuitive workflow.

There are still multiple limitations present in the current simulation environment. These will be mentioned and described in the *Next Steps* section.

5 Conclusion

The main purpose of this thesis was to provide interactive tools that would greatly facilitate the process of developing intuitions about core algorithms used in robotics. As we have seen, the set of interfaces implemented and described in this work accomplish this goal by allowing the user to visualize the algorithms and concepts in action while being able to directly interact and change parameters. Importantly, this environment allows the exploration of each concept *in isolation*, which is crucial in highly modular systems like robotic systems are by nature.

6 Next Steps

There are four main areas in which this work can be improved and expanded:

- **Performance**: Although multiple optimizations already implemented improved considerably the *frames per rate* and the image quality, there is still work to do to bring performance to a level comparable modern simple 2D games. One promising approach is to use blitting for faster rendering. Another one is to use the primitives of the gaming library to draw the figures from scratch instead of plotting them with a separate library and then converting the image to a valid format.

- **Scope**: The architecture was designed with modularity in mind so it would be easy to add extra interfaces without much effort. Importantly, the topic of *mapping* is not covered in the simulation environment. Therefore, creating an interface to visualize one of the most currently used methods in robotics called *Simultaneous Localization and Mapping* (SLAM) would be an excellent step forward. An example of an interface to illustrate SLAM would be to initialize a robot in a maze but this time it has to create a map of the environment, so

originally it only sees its local surroundings in a radius corresponding to the range of its sensors. There can be two windows next to each other, one with the map the robot is building and one with the actual map. The user would then be able to move the robot and observe the quality of the map being built by comparing it to the ground truth.

- **Integration**: The idea of first covering each topic individually with simple environments is important to develop conceptual building blocks, but the main goal is to understand a fully integrated robotic system, which has all its modules constantly running simultaneously and interacting with each other. To this end, a rich interface that integrates all the concepts previously covered is an important feature for the completeness of the simulation environment. As an example, one could create a city map where the user controls the parameters of an autonomous taxi and customers appear on random locations in the city. The user can select the localization method (e.g. Kalman Filters or Particle Filters), the smoothness parameters of the path, the cost of each action, etc... Once the path has been found and smoothed the car would use PID control to follow the given path, where the CTE and PID terms are visible on the screen.

- **Education**: This application currently assumes that the presented concepts *have already been studied*, and it offers an extra tool to understand them. Consequently, it cannot be used independently. The *About* section is meant to address this limitation to some degree, however it currently offers only a very short description of each concept. This section can be expanded to contain a compacted version of the explanations presented here, introducing some formulas and references where needed so the user can have not only a better idea of how each algorithm work but also access to free resources to deepen their theoretical understanding.

In order to assess qualitatively and quantitatively the usefulness of the simulation environment for its intended goal, it needs to be tested on a controlled setting. Specifically, a controlled study can be carried out with two groups of students. Both groups can receive the same educational material covering the underlying theory, a good candidate would be the course that originally motivated this work [16], then only one group would be given the simulation environment to play with and deepen their intuitions. Finally, a test can be given to both groups to test their knowledge and the experimental group would receive an additional questionnaire to report on their subjective experience with the application.

References

[1] S. Thrun, W. Burgard, and D. Fox. *Probabilistic Robotics*. Intelligent Robotics and Autonomous Agents series. MIT Press, 2005.

[2] Sandra Cairncross and Mike Mannion. Interactive multimedia and learning: Realizing the benefits. *Innovations in Education and Teaching International - INNOV EDUC TEACH INT*, 38:156–164, 05 2001.

[3] J. Veřmiřovský. The importance of visualisation in education. 2013.

[16] https://www.udacity.com/course/artificial-intelligence-for-robotics–cs373

[4] Andrew H. Jazwinski. *Stochastic processes and filtering theory.* Number 64 in Mathematics in science and engineering. Acad. Press, New York, NY [u.a.], 1970.

[5] Henrik. Seckler. 2d robotic mapping using a grid-based fast-slam algorithm with applications of reinforcement learning. 2020.

[6] Jarosław Majchrzak, Mateusz Michalski, and Grzegorz Wiczynski. Distance estimation with a long-range ultrasonic sensor system. *Sensors Journal, IEEE*, 9:767 – 773, 08 2009.

[7] Yi Cheng and Gong Wang. Mobile robot navigation based on lidar. pages 1243–1246, 06 2018.

[8] Andréa Macario Barros, Yoann Moline, Frédérick Carrel, Maugan Michel, and Gwenolé Corre. A comprehensive survey of visual slam algorithms. *Robotics*, 11, 02 2022.

[9] Zhao Gong, Songwen Jiang, Qizhi Meng, Yanlei Ye, Peng Li, Fugui Xie, Huichan Zhao, Chunzhe Lv, Xiaojie Wang, and Xinjun Liu. Shuyu robot: An automatic rapid temperature screening system. *Chinese Journal of Mechanical Engineering*, 33, 12 2020.

[10] Bo Chang, Renmei Xu, and Tiffany Watt. The impact of colors on learning. 06 2018.

[11] Dung Tran, Delowar Hossain, and Genci Capi. Robot localization in textureless environments based on multi-features image indexing. 02 2020.

[12] De Jong Yeong, Gustavo Velasco-Hernandez, John Barry, and Joseph Walsh. Sensor and sensor fusion technology in autonomous vehicles: A review. *Sensors*, 21(6), 2021.

[13] Steven M. LaValle. *Planning Algorithms.* Cambridge University Press, 2006.

[14] T Palaniyappan, Vaishali Yadav, Ruchira Singla, Vijay Tayal, and Pallavi Choudekar. Pid control design for a temperature control system. pages 632–637, 04 2018.

Selbstständigkeitserklärung

Hiermit versichere ich, dass die vorliegende Arbeit von mir selbstständig verfasst wurde und dass keine anderen als die angegebenen Quellen und Hilfsmittel benutzt wurden. Diese Erklärung erstreckt sich auch auf in der Arbeit enthaltene Graphiken, Zeichnungen, Kartenskizzen und bildliche Darstellungen.

Statement of Originality

I hereby confirm that I have written the accompanying thesis by myself, without contributions from any sources other than those cited in the text and acknowledgements. This applies also to all graphics, drawings, maps and images included in the thesis.